

# **RA4**

## **Ejercicios Obligatorios**

En **AZUL** los textos de los enunciados  
En **NEGRO** las respuestas

# ÍNDICE

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>Contexto.....</b>                                                           | <b>3</b>  |
| <b>Ejercicio 1: Monitorización del clúster Hadoop .....</b>                    | <b>3</b>  |
| Objetivo .....                                                                 | 3         |
| <b>Parte 1 — Arranque del clúster.....</b>                                     | <b>3</b>  |
| 1.1.1. Estructura actual del Cluster.....                                      | 3         |
| 1.1.2. Arquitectura del Host físico.....                                       | 5         |
| <b>Parte 2 — Monitorización web.....</b>                                       | <b>6</b>  |
| 1.2.1. NameNode UI (http://localhost:9870/) .....                              | 6         |
| 1.2.2. DataNode UI (http://localhost:9864 —1 y 2— http://localhost:9865) ..... | 8         |
| 1.2.3. YARN, ResourceManager UI (http://localhost:8088) .....                  | 10        |
| <b>Parte 3 — Prometheus y Grafana .....</b>                                    | <b>11</b> |
| 1.3.1. Prometheus (http://localhost:9090/).....                                | 11        |
| 1.3.2. Grafana (http://localhost:3000/).....                                   | 11        |
| 1.3.3. Ejecuta una consulta (ej: jmx_exporter_up) .....                        | 12        |
| 1.3.4. Crea una visualización .....                                            | 15        |
| 1.3.5. Configura una alerta .....                                              | 17        |
| <b>Parte 4 — Terminal.....</b>                                                 | <b>18</b> |
| Entrega .....                                                                  | 22        |
| <b>Ejercicio 2: Análisis y optimización de rendimiento en Hadoop.....</b>      | <b>22</b> |
| Contexto.....                                                                  | 22        |
| <b>1. Identificación de problemas.....</b>                                     | <b>22</b> |
| <b>2. Herramientas de monitorización .....</b>                                 | <b>24</b> |
| <b>3. Estrategia de solución .....</b>                                         | <b>24</b> |
| <b>4. Sistema de alertas .....</b>                                             | <b>25</b> |
| Entrega .....                                                                  | 26        |
| Conclusión .....                                                               | 27        |

## Contexto [-Ir al índice-](#)

En estos ejercicios trabajarás la **monitorización, análisis y optimización de sistemas Big Data**, concretamente sobre un clúster Hadoop. Se combinará una parte práctica con una parte de análisis de caso real.

## Ejercicio 1: Monitorización del clúster Hadoop [-Ir al índice-](#)

### Objetivo

Monitorizar un clúster Hadoop mediante tres métodos distintos: interfaces web nativas, herramientas de visualización (Grafana + Prometheus) y línea de comandos.

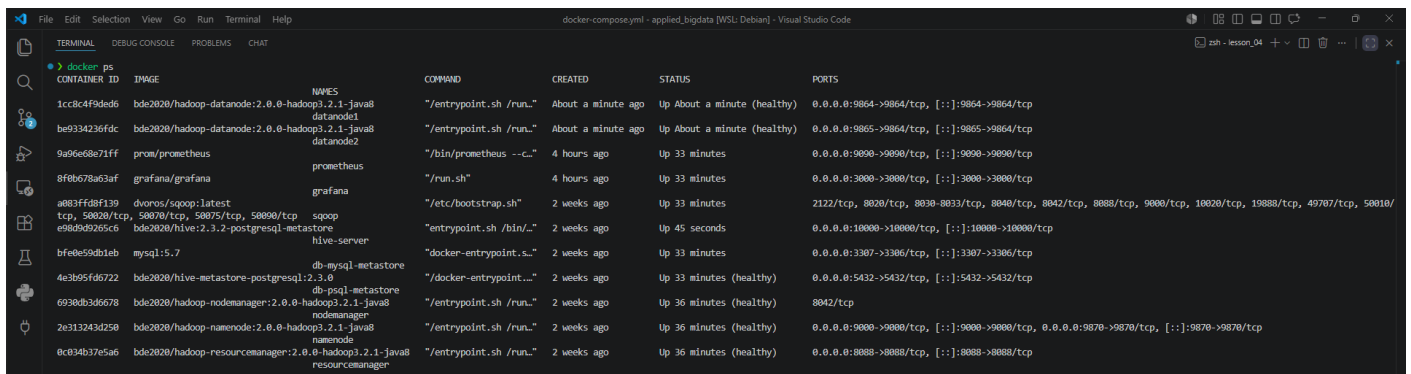
### Parte 1 — Arranque del clúster [-Ir al índice-](#)

- Descarga y pon en marcha el clúster usando Docker Compose.
- Verifica los siguientes contenedores:
  - Namenode
  - Datanode
  - Resourcemanager
  - Nodemanager
  - Grafana
  - Prometheus

#### 1.1.1 Estructura actual del Cluster [-Ir al índice-](#)

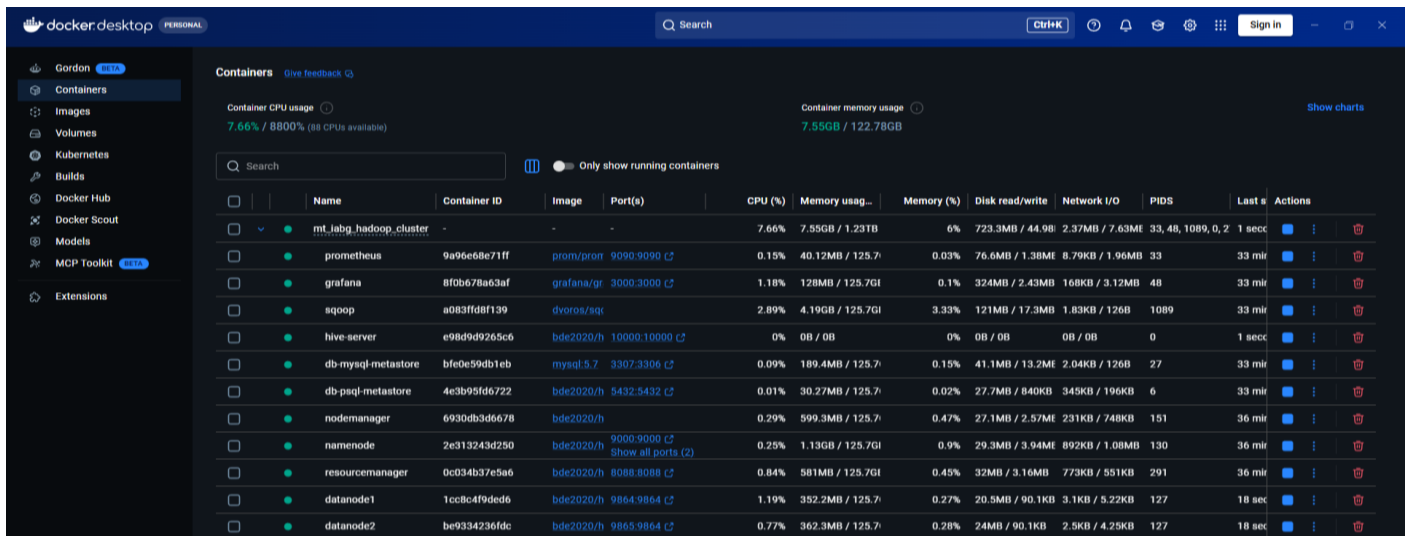
[docker-compose.yml](#) download from <https://github.com/dcueli-com> [develop]

#### ➤ docker ps



| CONTAINER ID  | IMAGE                                                  | NAMES              | COMMAND                   | CREATED            | STATUS                      | PORTS                                                                                                       |
|---------------|--------------------------------------------------------|--------------------|---------------------------|--------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------|
| 1cc8c4f9de9d  | bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8        | datanode1          | "/entrypoint.sh /run..."  | About a minute ago | Up About a minute (healthy) | 0.0.0.0:9864->9864/tcp, [::]:9864->9864/tcp                                                                 |
| be9334236fdc  | bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8        | datanode2          | "/entrypoint.sh /run..."  | About a minute ago | Up About a minute (healthy) | 0.0.0.0:9865->9864/tcp, [::]:9865->9864/tcp                                                                 |
| 9a96e68e71ff  | prom/prometheus                                        | prometheus         | "/bin/prometheus --c..."  | 4 hours ago        | Up 33 minutes               | 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp                                                                 |
| 8f6b678a63af  | grafana/grafana                                        | grafana            | "/run.sh"                 | 4 hours ago        | Up 33 minutes               | 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp                                                                 |
| a083ffdf8f139 | dvoros/sqoop:latest                                    | sqoop              | "/etc/bootstrap.sh"       | 2 weeks ago        | Up 33 minutes               | 2122/tcp, 8020/tcp, 8030-8033/tcp, 8040/tcp, 8088/tcp, 9000/tcp, 10020/tcp, 19888/tcp, 49707/tcp, 50018/tcp |
| e98d9d9265c6  | bde2020/hive:2.3.2-postgresql-metastore                | hive-server        | "/entrypoint.sh /bin/..." | 2 weeks ago        | Up 45 seconds               | 0.0.0.0:10000->10000/tcp, [::]:10000->10000/tcp                                                             |
| bfe0e59db1eb  | mysql:5.7                                              | db-mysql-metastore | "/docker-entrypoint.s..." | 2 weeks ago        | Up 33 minutes               | 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp                                                                 |
| 4e3b95fd6722  | bde2020/hive-metastore-postgresql:2.3.0                | db-psql-metastore  | "/docker-entrypoint..."   | 2 weeks ago        | Up 33 minutes (healthy)     | 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp                                                                 |
| 6930db3d6678  | bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8     | nodemanager        | "/entrypoint.sh /run..."  | 2 weeks ago        | Up 36 minutes (healthy)     | 8042/tcp                                                                                                    |
| 2e313243d250  | bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8        | namenode           | "/entrypoint.sh /run..."  | 2 weeks ago        | Up 36 minutes (healthy)     | 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp                    |
| 0c034b37e5a6  | bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8 | resourcemanager    | "/entrypoint.sh /run..."  | 2 weeks ago        | Up 36 minutes (healthy)     | 0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp                                                                 |

Figura 1.1



| Name                   | Container ID  | Image      | Port(s)     | CPU (%) | Memory usage     | Memory (%) | Disk read/write | Network I/O     | PIDS               | Last s | Actions                   |
|------------------------|---------------|------------|-------------|---------|------------------|------------|-----------------|-----------------|--------------------|--------|---------------------------|
| mt_labg.hadoop_cluster | -             | -          | -           | 7.66%   | 7.55GB / 1.23TB  | 6%         | 723.3MB / 44.9B | 2.37MB / 7.63MB | 33, 48, 1089, 0, 2 | 1 sec  | [Stop] [Refresh] [Delete] |
| prometheus             | 9a96e68e71ff  | prom/prom  | 9090:9090   | 0.15%   | 40.12MB / 125.7  | 0.03%      | 76.6MB / 1.38MB | 8.79KB / 1.96MB | 33                 | 33 mi  | [Stop] [Refresh] [Delete] |
| grafana                | 8f6b678a63af  | grafana/gr | 3000:3000   | 1.18%   | 128MB / 125.7Gi  | 0.1%       | 324MB / 2.43MB  | 168KB / 3.12MB  | 48                 | 33 mi  | [Stop] [Refresh] [Delete] |
| sqoop                  | a083ffdf8f139 | dvoros/sq  |             | 2.89%   | 4.19GB / 125.7Gi | 3.33%      | 121MB / 17.3MB  | 1.83KB / 126B   | 1089               | 33 mi  | [Stop] [Refresh] [Delete] |
| hive-server            | e98d9d9265c6  | bde2020/h  | 10000:10000 | 0%      | 0B / 0B          | 0%         | 0B / 0B         | 0B / 0B         | 0                  | 1 sec  | [Stop] [Refresh] [Delete] |
| db-mysql-metastore     | bfe0e59db1eb  | mysql:5.7  | 3307:3306   | 0.09%   | 189.4MB / 125.7  | 0.15%      | 41.1MB / 13.2MB | 2.04KB / 126B   | 27                 | 33 mi  | [Stop] [Refresh] [Delete] |
| db-psql-metastore      | 4e3b95fd6722  | bde2020/h  | 5432:5432   | 0.01%   | 30.27MB / 125.7  | 0.02%      | 27.7MB / 840KB  | 345KB / 196KB   | 6                  | 33 mi  | [Stop] [Refresh] [Delete] |
| nodemanager            | 6930db3d6678  | bde2020/h  |             | 0.29%   | 599.3MB / 125.7  | 0.47%      | 27.1MB / 2.57MB | 231KB / 748KB   | 151                | 36 mi  | [Stop] [Refresh] [Delete] |
| namenode               | 2e313243d250  | bde2020/h  | 9000:9000   | 0.25%   | 1.13GB / 125.7Gi | 0.9%       | 29.3MB / 3.94MB | 892KB / 1.08MB  | 130                | 36 mi  | [Stop] [Refresh] [Delete] |
| resourcemanager        | 0c034b37e5a6  | bde2020/h  | 8088:8088   | 0.84%   | 581MB / 125.7Gi  | 0.45%      | 32MB / 3.16MB   | 773KB / 551KB   | 291                | 36 mi  | [Stop] [Refresh] [Delete] |
| datanode1              | 1cc8c4f9de9d  | bde2020/h  | 9864:9864   | 1.19%   | 352.2MB / 125.7  | 0.27%      | 20.5MB / 90.1KB | 3.1KB / 5.22KB  | 127                | 18 sec | [Stop] [Refresh] [Delete] |
| datanode2              | be9334236fdc  | bde2020/h  | 9865:9864   | 0.77%   | 362.3MB / 125.7  | 0.28%      | 24MB / 90.1KB   | 2.9KB / 4.29KB  | 127                | 18 sec | [Stop] [Refresh] [Delete] |

Figura 1.2

Con la información de **docker ps**, la arquitectura del entorno queda como sigue:

```
Docker Compose
├── Hadoop HDFS
│   ├── NameNode
│   │   ├── Contenedor: namenode
│   │   ├── Imagen: bde2020/hadoop-namenode
│   │   ├── Puertos:
│   │   │   ├── 9870 → Web UI HDFS
│   │   │   └── 9000 → Servicio HDFS
│   │   └── Estado: healthy
│   ├── DataNode1
│   │   ├── Contenedor: datanode1
│   │   ├── Imagen: bde2020/hadoop-datanode
│   │   ├── Puerto:
│   │   │   └── 9864 → Web UI DataNode
│   │   └── Estado: healthy
│   └── DataNode2
│       ├── Contenedor: datanode2
│       ├── Imagen: bde2020/hadoop-datanode
│       ├── Puerto:
│       │   └── 9865 → Web UI DataNode
│       └── Estado: healthy
├── YARN
│   ├── ResourceManager
│   │   ├── Contenedor: resourcemanager
│   │   ├── Puerto:
│   │   │   └── 8088 → Web UI YARN
│   │   └── Estado: healthy
│   └── NodeManager
│       ├── Contenedor: nodemanager
│       ├── Puerto interno:
│       │   └── 8042
│       └── Estado: healthy
├── Hive
│   ├── Hive Server
│   │   ├── Contenedor: hive-server
│   │   ├── Puerto:
│   │   │   └── 10000 → HiveServer2
│   │   └── Estado: running
│   └── PostgreSQL Metastore
│       ├── Contenedor: db-psql-metastore
│       ├── Puerto:
│       │   └── 5432 → PostgreSQL
│       └── Estado: healthy
├── Sqoop
│   ├── Contenedor: sqoop
│   ├── Imagen: davoros/sqoop
│   └── Estado: running
├── MySQL
│   ├── Contenedor: db-mysql-metastore
│   ├── Puerto:
│   │   └── 3307 → MySQL
│   └── Estado: running
├── Prometheus
│   ├── Contenedor: prometheus
│   ├── Puerto:
│   │   └── 9090 → Métricas
│   └── Estado: running
└── Grafana
    ├── Contenedor: grafana
    ├── Puerto:
    │   └── 3000 → Dashboards
    └── Estado: running
```

**Prometheus** queda accesible en el puerto **9090**  
<http://localhost:9090>

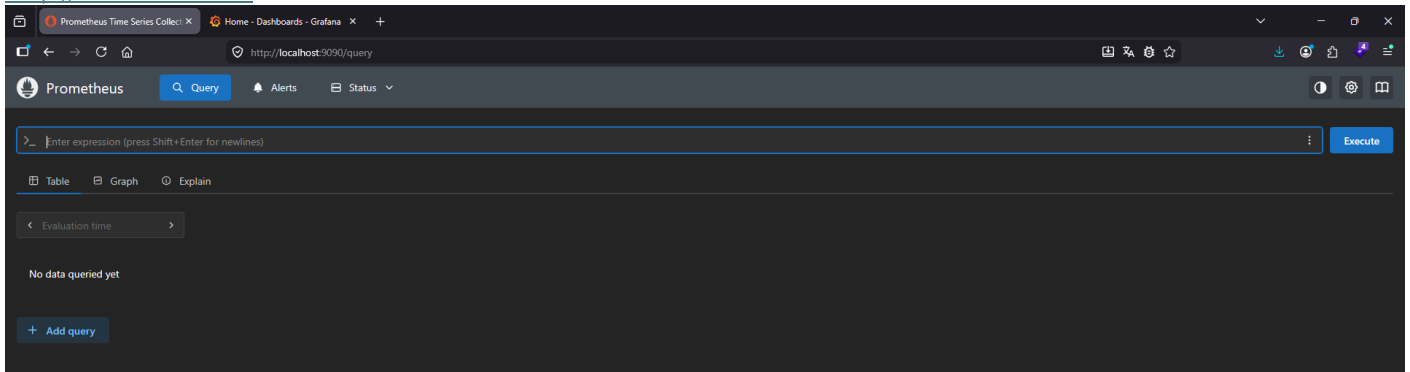


Figura 1.3

**Grafana** queda accesible en el puerto **3000**  
<http://localhost:3000>

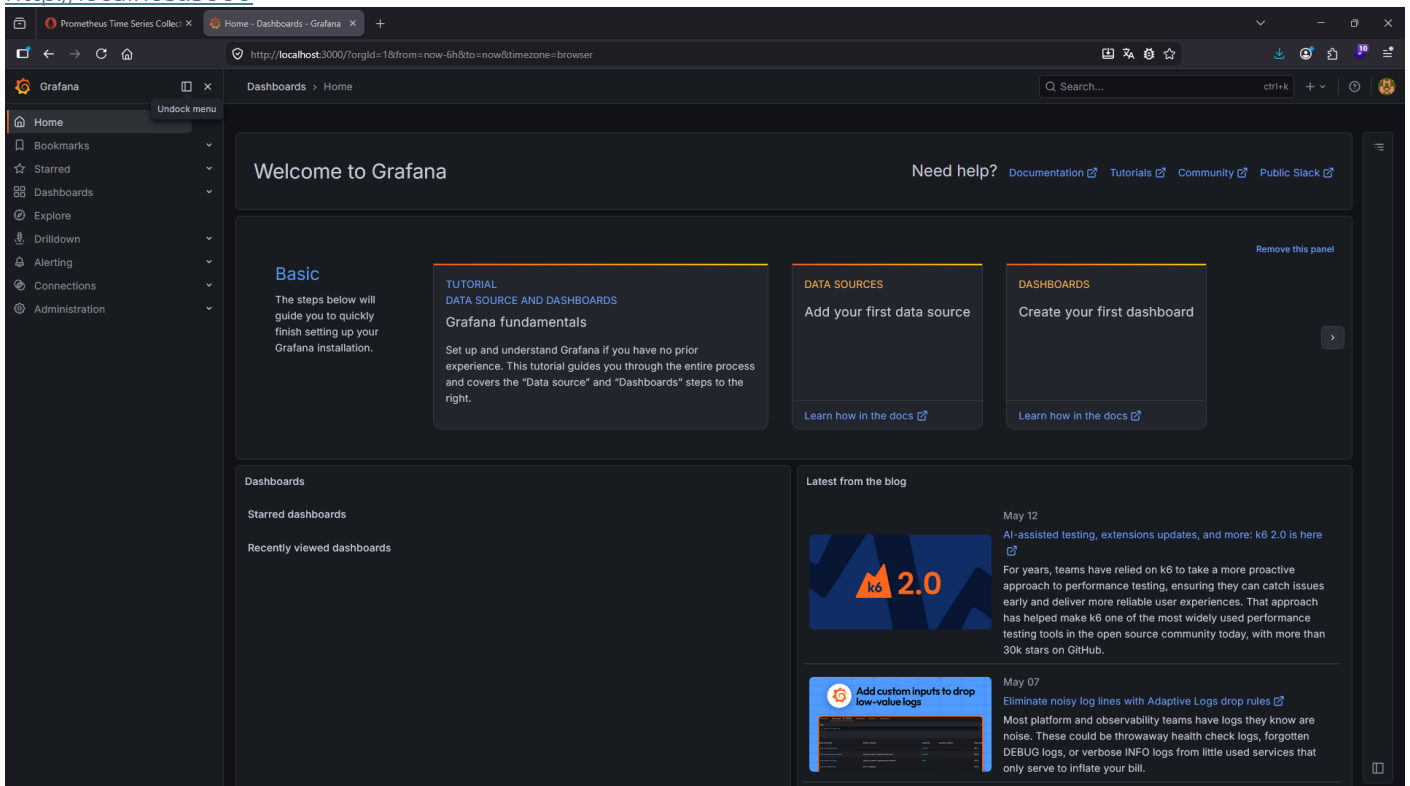


Figura 1.4

### 1.1.2. Arquitectura del Host físico -Ir al índice-

El dimensionamiento del clúster se ha establecido bajo la siguiente línea base del hardware.

#### Capacidad de cómputo

Dos procesadores en arquitectura Intel(R) Xeon(R) CPU E5-2699 v4 a 2.20GHz, optimizada para cargas de trabajo multihilo e integración paralela.

#### Memoria Volátil

256 GB de memoria RAM instalada.

#### GPU

NVIDIA GeForce RTX 3060 (12 GB).

#### Almacenamiento Físico

Matriz de almacenamiento con 4.16 TB totales, con un volumen de ocupación registrado del 39.9% (1.66 TB usados).

## Parte 2 — Monitorización web [-Ir al índice-](#)

- Accede y captura.
- Describe qué información muestra cada una

### 1.2.1 NameNode UI (<http://localhost:9870/>) [-Ir al índice-](#)

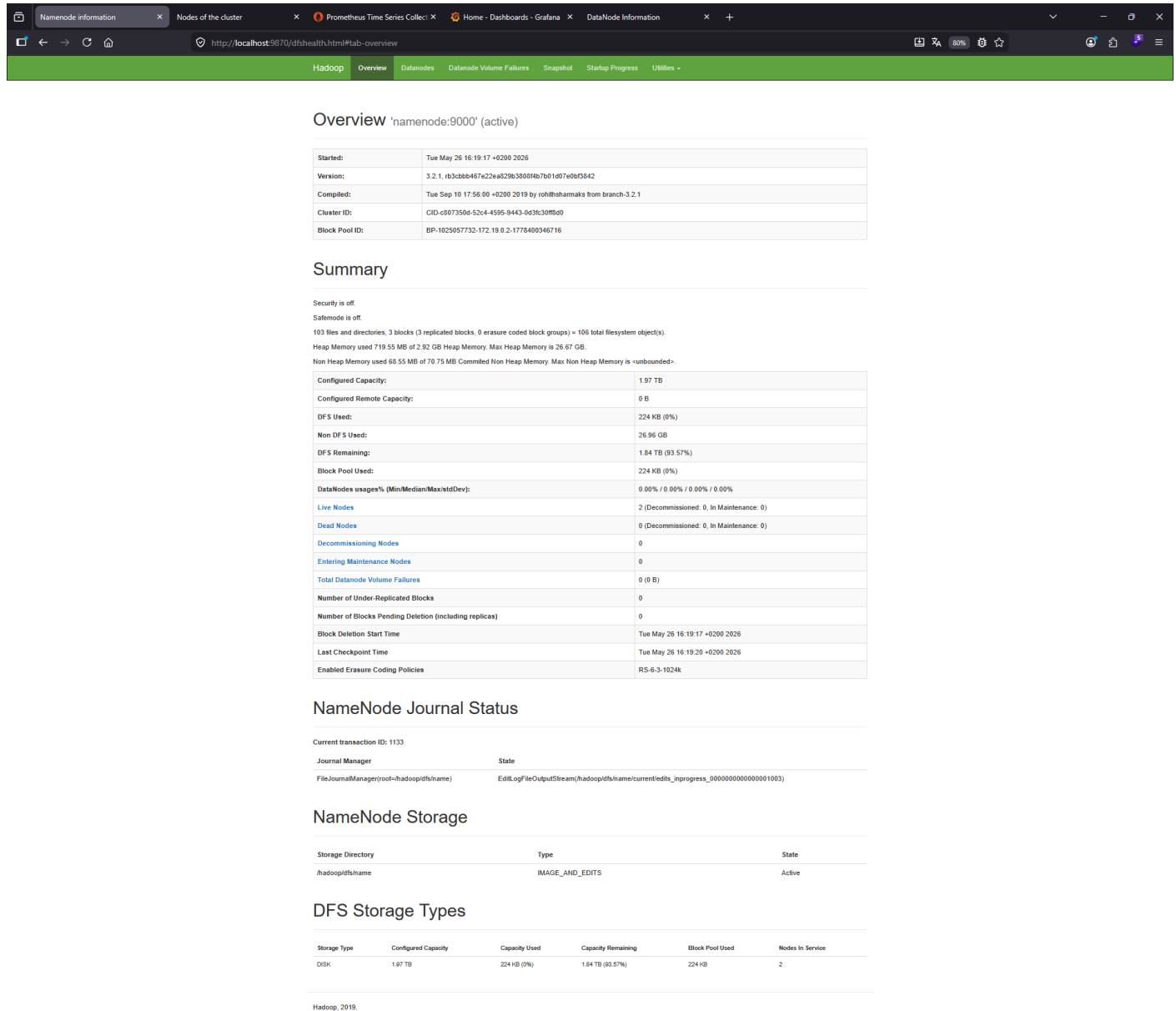


Figura 2.1

Se accede a la UI del NameNode activo del Clúster de Hadoop mediante:

<http://localhost:9870>

Como se recoge en la captura de pantalla ([Fig. 2.1](#)), la interfaz muestra el estado general del sistema HDFS donde se incluye el almacenamiento distribuido, bloques, nodos activos, uso de memoria, replicación y estado interno del NameNode.

### 1.2.1.1 Estado general

El sistema utiliza el almacenamiento distribuido gestionando el espacio de varios servidores para trabajar como si fuera un único disco gigante.

- Estado del NameNode  
Activo (**active**), operativo, sin errores de replicación, con el clúster funcionando correctamente.
  - Security** deshabilitado.
  - Safemode** desactivado.HDFS está plenamente operativo y permitiendo operaciones normales de lectura y escritura.
- Archivos, bloques y replicación  
Registrados **65 archivos y directorios**, repartidos físicamente en **3 bloques** de datos con **3 bloques replicados y sub-replicados**, sin riesgo de pérdida de información.

| Métrica                 | Valor |
|-------------------------|-------|
| Archivos/directorios    | 65    |
| Bloques totales         | 3     |
| Bloques replicados      | 3     |
| Under-Replicated Blocks | 0     |

Todos los bloques del sistema están correctamente replicados, no existiendo incidencias de redundancia y manteniendo la tolerancia a fallos.

### 1.2.1.2. Capacidad y espacio disponible

Al revisar los contadores de espacio en disco, los resultados son los siguientes:

- Capacidad configurada** con un espacio total de 1.97 TB.
- Espacio usado por el clúster (DFS Used)** actualmente es de 224 KB (el 0%) lo que se intuye que es el valor mínimo indicando que el sistema está limpio y listo para empezar a recibir trabajar.
- Espacio usado por otros programas (Non DFS Used) es de 26.95 GB**, entre el OS y los contenedores no pudiendo usar este espacio.
- Espacio libre restante (DFS Remaining)** para almacenar toda la información que sea necesaria es de 1.84 TB libres (un 93.57% del total).

| Métrica                     | Valor          |
|-----------------------------|----------------|
| Capacidad total configurada | <b>1.97 TB</b> |
| Espacio usado por HDFS      | <b>224 KB</b>  |
| Espacio libre               | <b>1.84 TB</b> |
| Uso HDFS                    | <b>0%</b>      |

Lo que se puede interpretar aquí por las capacidades, es que el clúster se encuentra prácticamente vacío y sin carga real de datos, deduciendo que se trata de un entorno de pruebas (como es este caso).

### 1.2.1.3 Estado de memoria del NameNode

El uso de Memoria del NameNode actual es de 821.63 MB de una reserva inicial de 2.92 GB (con un límite asignable de hasta 26.67 GB).

| Tipo memoria      | Uso              |
|-------------------|------------------|
| Heap Memory usada | <b>821.63 MB</b> |
| Heap máxima       | <b>26.67 GB</b>  |
| Non-Heap usada    | <b>71.64 MB</b>  |

El NameNode tiene un consumo de memoria muy bajo respecto al máximo disponible, por lo que no existen síntomas de saturación ni presión de memoria.

### 1.2.1.4. Estado de los DataNodes

| Métrica           | Valor |
|-------------------|-------|
| Live Nodes        | 2     |
| Dead Nodes        | 0     |
| Fallos de volumen | 0     |

### 1.2.1.5 Estado interno y journaling

```
/hadoop/dfs/name
```

```
/hadoop/dfs/name/current/edits_inprogress_0000000000000001003
```

Los DataNodes están correctamente conectados, no existen errores de replicación, no hay nodos caídos, y el sistema dispone de capacidad suficiente de almacenamiento y memoria para soportar carga adicional.

|                    |                                                 |
|--------------------|-------------------------------------------------|
| <b>Cluster ID:</b> | CID-c807350d-52c4-4595-9443-0d3fc30ff8d0        |
| <b>Version:</b>    | 3.2.1, rb3cbb467e22ea829b3808f4b7b01d07e0bf3842 |

| namenode address | block pool id                          | actor state | last heartbeat | last block report | last block report size (max size) |
|------------------|----------------------------------------|-------------|----------------|-------------------|-----------------------------------|
| namenode:9000    | BP-1025057732-172.19.0.2-1778400346716 | RUNNING     | 0s             | 35 minutes        | 30 B (64 MB)                      |

| Directory        | StorageType | Capacity Used | Capacity Left | Capacity Reserved | Reserved Space for Replicas | Blocks |
|------------------|-------------|---------------|---------------|-------------------|-----------------------------|--------|
| /hadoop/dfs/data | DISK        | 112 KB        | 942.22 GB     | 0 B               | 0 B                         | 3      |

[dcueli.com](http://dcueli.com)  
[linkedin.com/in/dcueli](https://www.linkedin.com/in/dcueli)  
Página 8 | 27



**DataNode2** queda accesible en el puerto **9865**, según configuración en el [docker-compose.yml](#) <http://localhost:9865>

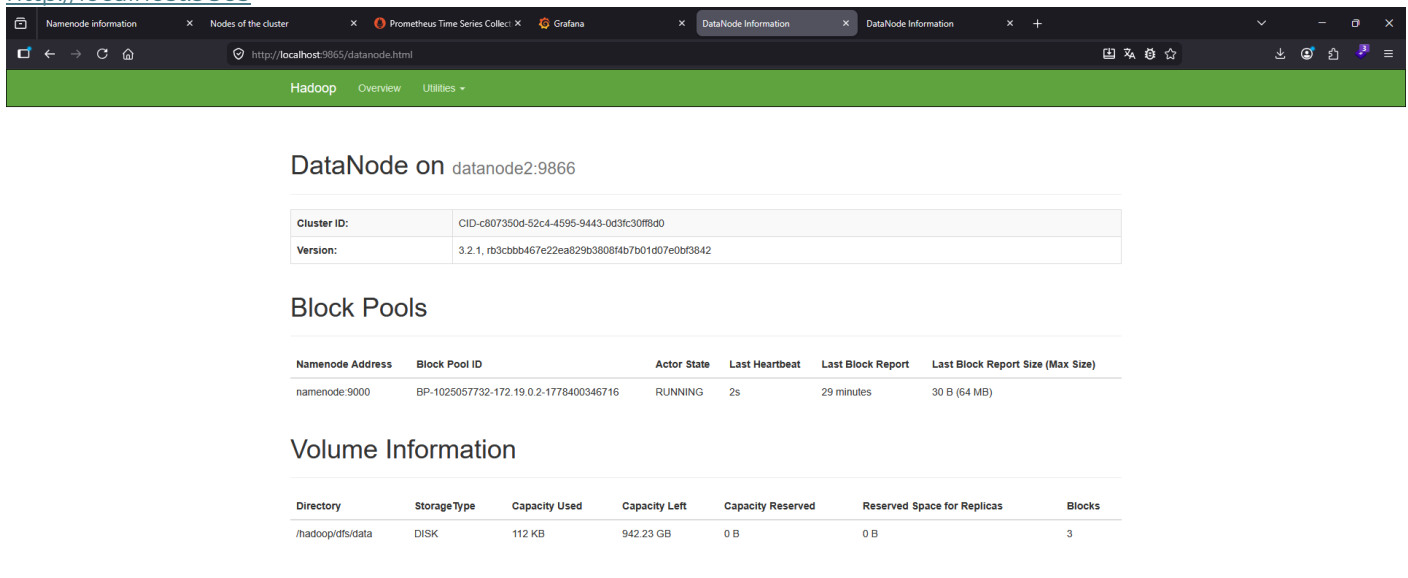


Figura 2.3

Para no extender el informe, no necesitamos repetir la información que ya tenemos del [NameNode](#), sino centrarnos en lo que cada DataNode informa individualmente, por lo que nos vamos a enfocar en la **persistencia** y **salud** de los nodos esclavos que es donde se almacenan físicamente los bloques de datos e informar de su estado al maestro.

### 1.2.2.1. Estado Operativo y Comunicación

Ambos nodos (**datanode1** y **datanode2**) mantienen una conexión activa (**running**) con el NameNode (**namenode:9000**) y han informado correctamente su estructura de bloques, con una latencia de 35 y 29 minutos respectivamente, lo que confirma que el ciclo de datos está funcionando. Los tiempos de respuesta (*heartbeat*) son “inmediatos”, lo que garantiza que NameNode conoce el estado de los esclavos en “tiempo real”.

| datanode1           |                |
|---------------------|----------------|
| Métrica             | Valor          |
| Estado              | <b>running</b> |
| Versión Hadoop      | <b>3.2.1</b>   |
| Último Block Report | <b>35 min.</b> |
| Tamaño              | <b>30 B</b>    |
| Último heartbeat    | <b>0 s</b>     |

| datanode2           |                |
|---------------------|----------------|
| Métrica             | Valor          |
| Estado              | <b>running</b> |
| Versión Hadoop      | <b>3.2.1</b>   |
| Último Block Report | <b>29 min.</b> |
| Tamaño              | <b>30 B</b>    |
| Último heartbeat    | <b>2 s</b>     |

### 1.2.2.2. Capacidad y Persistencia de Volumen

Ambos nodos registran un uso mínimo de **112 KB**, reflejando un estado de equilibrio en la distribución de bloques iniciales y cada uno dispone de aproximadamente **942 GB** de espacio libre (**Capacity Left**). Gestionan físicamente **3 bloques** de datos en su directorio local (`/hadoop/dfs/data`) utilizando discos de tipo estándar (DISK), sin reservas de espacio adicionales para réplicas pendientes (*Reserved Space for Replicas: 0 B*), lo cual es correcto si se sigue deduciendo que es un entorno de pruebas (como es este caso).

| datanode1           |                  |
|---------------------|------------------|
| Métrica             | Valor            |
| Espacio usado       | <b>112 KB</b>    |
| Espacio libre       | <b>942.22 GB</b> |
| Bloques almacenados | <b>3</b>         |

| datanode2           |                  |
|---------------------|------------------|
| Métrica             | Valor            |
| Espacio usado       | <b>112 KB</b>    |
| Espacio libre       | <b>942.23 GB</b> |
| Bloques almacenados | <b>3</b>         |

### 1.2.2.3. Conclusión técnica

Ambos DataNodes se encuentran correctamente integrados en el clúster HDFS y operativos, sin errores de almacenamiento y bloques, ni incidencias de comunicación y con capacidad suficiente disponible.

### 1.2.3 YARN, ResourceManager UI (<http://localhost:8088>) -Ir al índice-

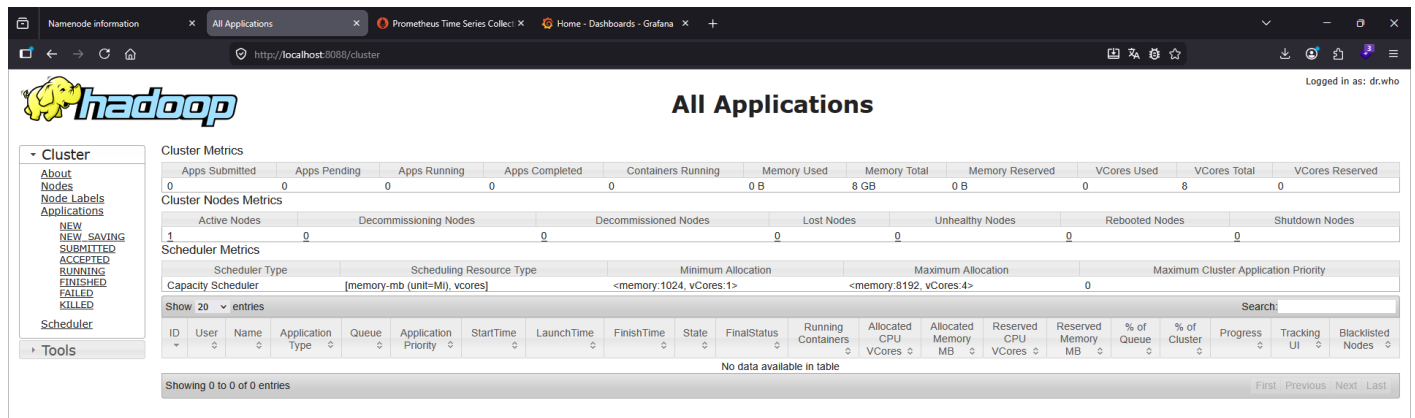


Figura 2.4

Se accede a la UI del NameNode activo del Clúster de Hadoop mediante:

<http://localhost:8088>

#### 1.2.3.1. Estado general

| Métrica            | Valor |
|--------------------|-------|
| Apps Submitted     | 0     |
| Apps Running       | 0     |
| Containers Running | 0     |
| Memory Used        | 0 B   |
| Memory Total       | 8 GB  |
| VCores Used        | 0     |
| VCores Total       | 8     |

Podemos interpretar estos valores como que el clúster, en ese momento, no estaba ejecutando trabajos distribuidos, no tenía aplicaciones MapReduce activas y todos los recursos permanecían disponibles.

Y como hemos comentado anteriormente esto es correcto para un entorno de pruebas, recién iniciado y sin carga de procesamiento activa.

#### 1.2.3.2. Nodos de Procesamiento Activos

| Métrica         | Valor |
|-----------------|-------|
| Active Nodes    | 1     |
| Lost Nodes      | 0     |
| Unhealthy Nodes | 0     |
| Shutdown Nodes  | 0     |

El sistema cuenta con **1 Nodo Activo** ejecutando el rol de *NodeManager*, listo para recibir y procesar tareas, con los indicadores de alerta a **0**, confirmando que la infraestructura es estable y no presenta fallos de hardware o red.

#### 1.2.3.3. Carga de Trabajo y Aplicaciones

La tabla de aplicaciones muestra:

No data available in table

Los contadores de aplicaciones (enviadas, pendientes, en ejecución y completadas) están a **0**. No hay ningún contenedor de trabajo activo y el listado de aplicaciones se encuentra vacío. Comportamiento correcto para un clúster que está en reposo y a la espera.

#### 1.2.3.4. Reglas del Planificador (Scheduler)

| Métrica        | Valor                     |
|----------------|---------------------------|
| Scheduler Type | <b>Capacity Scheduler</b> |
| Memoria mínima | <b>1024 MB</b>            |
| Memoria máxima | <b>8192 MB</b>            |
| vCores máximos | <b>4</b>                  |

El clúster está configurado bajo el modelo **Capacity Scheduler**, diseñado para garantizar recursos de forma predecible y evitar cuellos de botella.

Las reglas de asignación por cada tarea lanzada son:

- Para la **Memoria mínima**, asignada a cualquier tarea pequeña, que se envíe al sistema reservará como mínimo, **1 GB de RAM (1024 MB)** y **1 núcleo virtual (vCore)**.
- Para la **Memoria máxima**, evitará que un solo proceso ocupe la capacidad del clúster, por lo que el Planificador le da una asignación máxima a **8 GB de RAM** y **4 núcleos virtuales (vCores)**.

#### 1.2.3.5. Conclusión técnica

Con la información aportada por la UI del ResourceManager podemos concluir el análisis con que el sistema YARN se encuentra operativo y no existen nodos perdidos ni errores de salud así como que todos los recursos del clúster permanecen disponibles al no existir cargas activas.

Finalmente la configuración observada indica que es un entorno pequeño de pruebas, con recursos limitados intencionadamente, adecuado para el desarrollo de prácticas de procesamiento ligero.

## Parte 3 — Prometheus y Grafana [-Ir al índice-](#)

### *Incluye capturas y explicación*

#### 1.3.1 Prometheus (<http://localhost:9090/>) [-Ir al índice-](#)

**Prometheus** queda accesible en el puerto **9090**

<http://localhost:9090>

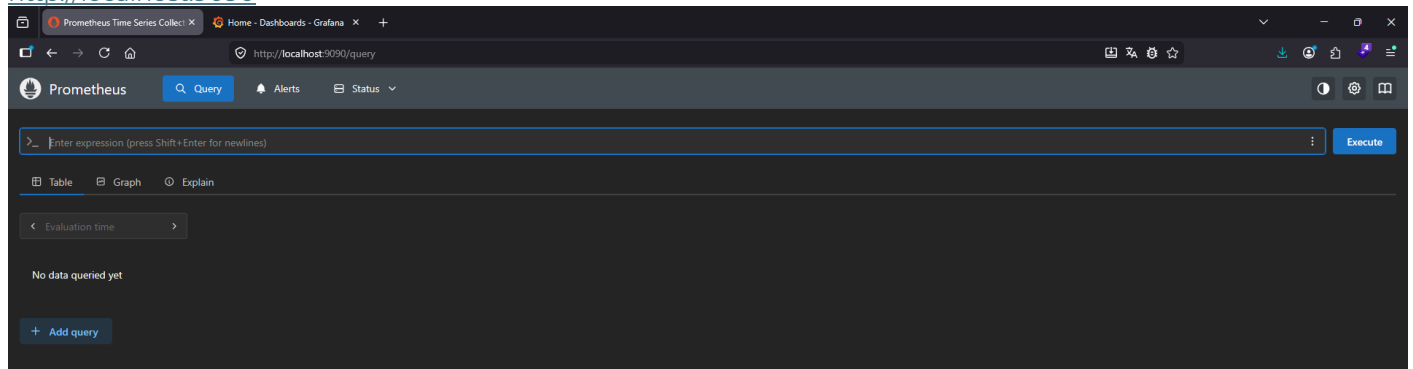


Figura 3.1

#### 1.3.2 Grafana (<http://localhost:3000/>) [-Ir al índice-](#)

**Grafana** queda accesible en el puerto **3000**

<http://localhost:3000>

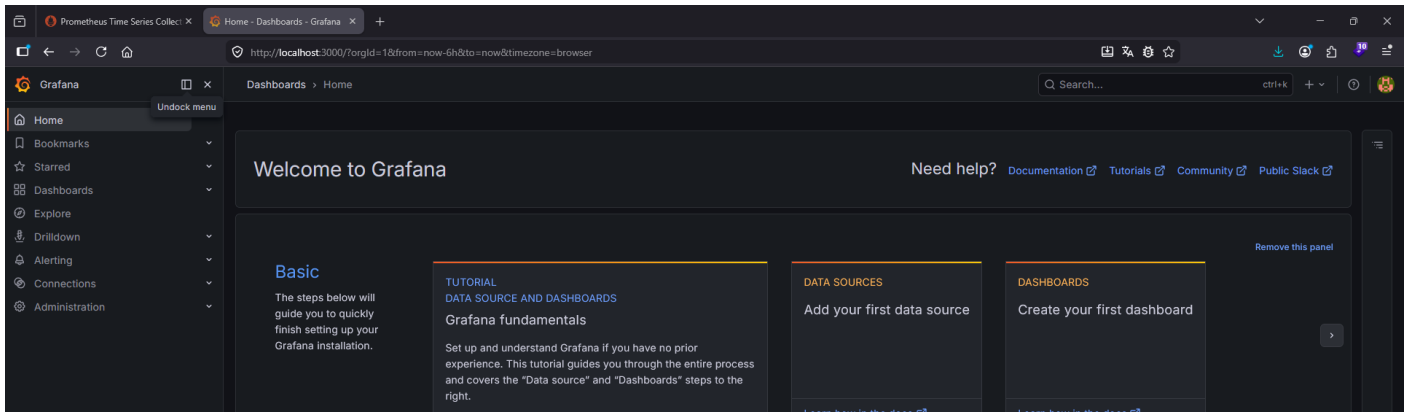


Figura 3.2

### 1.3.3 Ejecuta una consulta (ej: jmx\_exporter\_up) -Ir al índice-

Primeramente vamos a comprobar que Prometheus verdaderamente está operativo con la consulta más básica, **up** (cogeremos la captura de pantalla de la pestaña **Explain**).

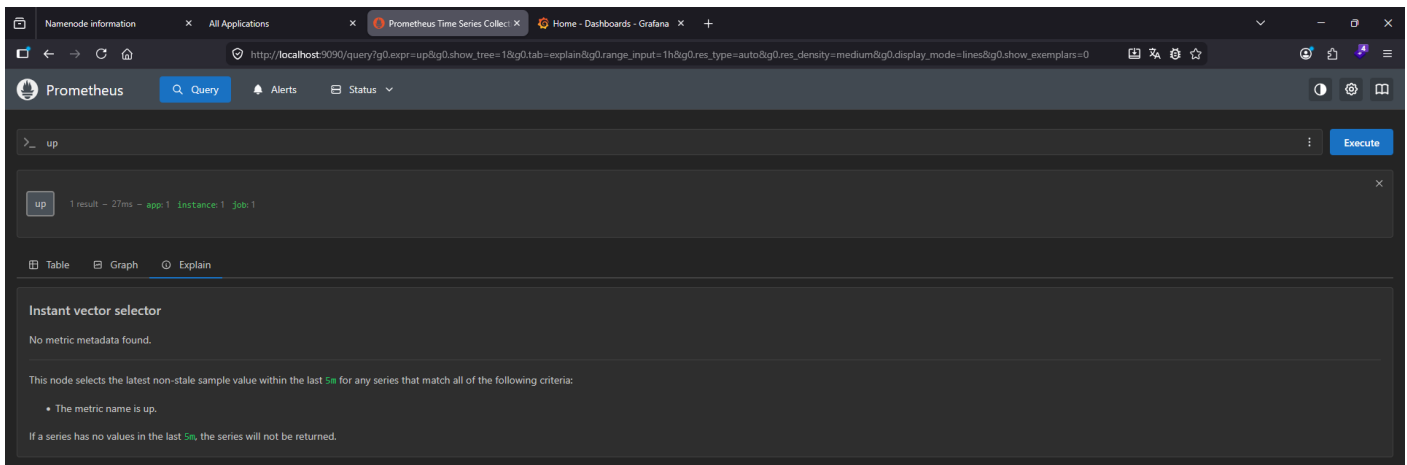


Figura 3.3

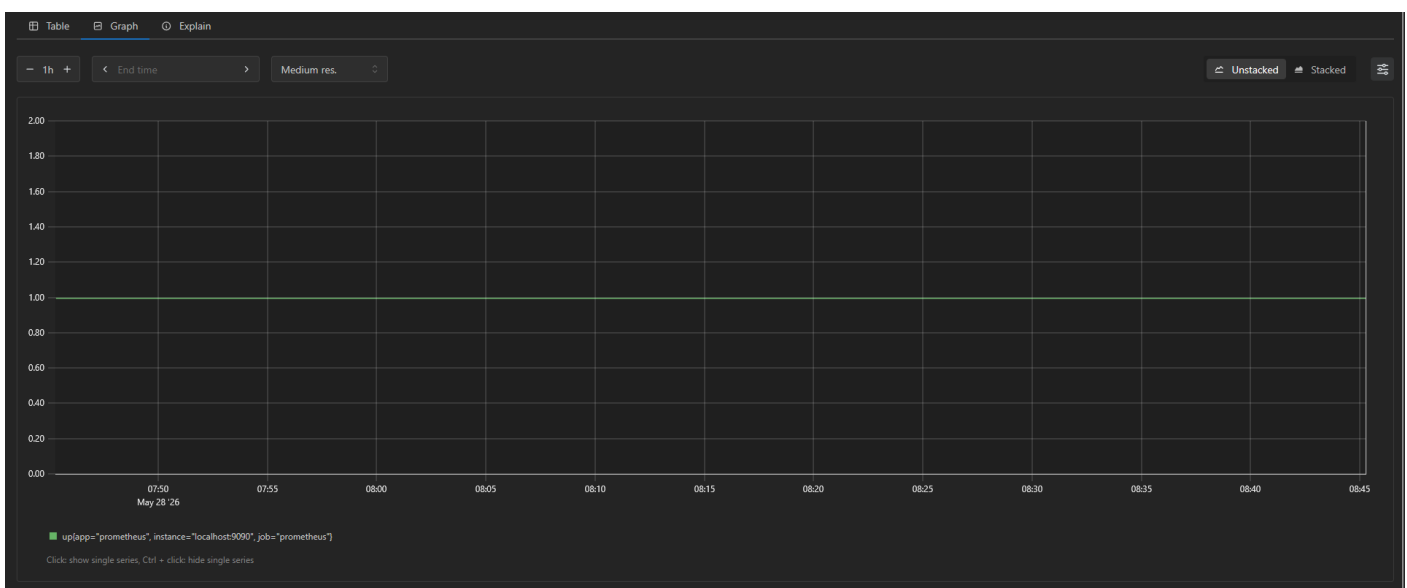


Figura 3.4

En la pestaña **Graph** (Fig. 3.4), podemos ver la línea recta continua en el valor 1, demostrando el tiempo de actividad del servicio en los últimos minutos.

Una vez que hemos comprobado que Prometheus funciona correctamente, pasamos a ejecutar consultas con JMX Prometheus Java Agent v0.20.0, pero para ello debemos integrarlos en los contenedores maestro y esclavos, NameNode y DataNode, respectivamente.

El agente fue agregado en las JVM de Hadoop mediante la:

- descarga del archivo binario de Java (**jmx\_prometheus\_javaagent.jar**) y guardándolo en el directorio de proyecto.

```
> wget https://repo1.maven.org/maven2/io/prometheus/jmx/jmx_prometheus_javaagent/0.20.0/jmx_prometheus_javaagent-0.20.0.jar -O ./jmx_prometheus_javaagent.jar
```

- creación de un fichero de configuración **jmx\_config.yml** para el agente de Java, aunque con una configuración vacía ya que no nos va a ser necesario de momento.
- adición de ambos ficheros a los volúmenes de los contenedores de Hadoop que se desean monitorizar (namenode, datanode1 y datanode2) en el archivo [docker-compose.yml](#)

```
[...]
volumes:
  - [...]
  - ./jmx_prometheus_javaagent.jar:/opt/jmx_exporter/jmx_prometheus_javaagent.jar
  - ./jmx_config.yml:/opt/jmx_exporter/jmx_config.yml
[...]
```

- configuración de la variable **HADOOP\_OPTS** en **hadoop.env** para arrancar Java con el agente JMX:

```
HADOOP_OPTS="-javaagent:/opt/jmx_exporter/jmx_prometheus_javaagent.jar=9010:/opt/jmx_exporter/jmx_config.yml"
```

- asignación a los puertos **9010**, **9011**, **9012** de los contenedores Hadoop (namenode, datanode1 y datanode1, respectivamente) para permitir que Prometheus pueda acceder a las métricas:

```
[...]
ports:
  - [...]
  - "9010:9010"
  - [...]
[...]
```

- configuración de **prometheus.yml** para añadir los nodos Hadoop como targets monitorizados:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'hadoop-jmx'
    static_configs:
      - targets: ['namenode:9010', 'datanode1:9010', 'datanode2:9010']
```

- recreación y aplicación de los cambios al clúster:

```
> docker compose up -d --force-recreate namenode datanode1 datanode2 prometheus
```

- verificación desde dentro del contenedor **NameNode** que el agente Java se está ejecutando correctamente:

```
> docker exec -it namenode bash
root@fba8a331269b:/# jps -v
101 NameNode -Dproc_namenode -
javaagent:/opt/jmx_exporter/jmx_prometheus_javaagent.jar=9010:/opt/jmx_exporter/jmx_config.yml -
Dhdfs.audit.logger=INFO,NullAppender -DHadoop.security.logger=INFO,RFAS -Dyarn.log.dir=/opt/hadoop-3.2.1/logs -
Dyarn.log.file=hadoop.log -Dyarn.home.dir=/opt/hadoop-3.2.1 -Dyarn.root.logger=INFO,console -
Djava.library.path=/opt/hadoop-3.2.1/lib/native -DHadoop.log.dir=/opt/hadoop-3.2.1/logs -DHadoop.log.file=hadoop.log
-Dhadoop.home.dir=/opt/hadoop-3.2.1 -DHadoop.id.str=root -DHadoop.root.logger=INFO,console -
DHadoop.policy.file=hadoop-policy.xml
2105 Jps -Dapplication.home=/usr/lib/jvm/java-8-openjdk-amd64 -Xms8m
root@fba8a331269b:/# _
```

- comprobación en Prometheus (<http://localhost:9090/targets>) que todos los targets aparecen en estado **UP**:

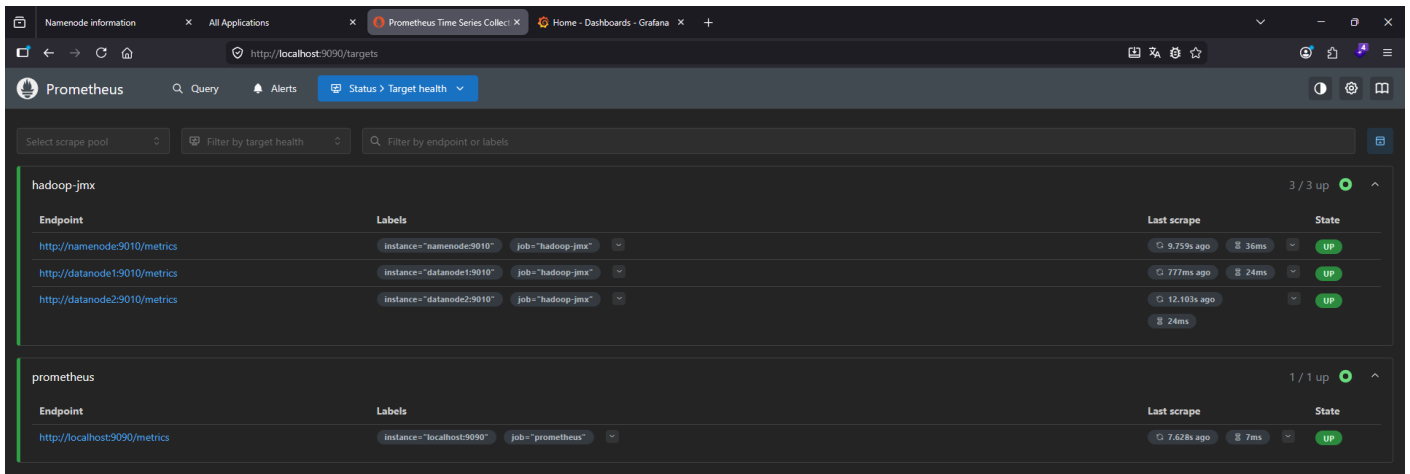


Figura 3.5

- confirmación que Prometheus ya puede recolectar métricas Hadoop mediante consultas como: **up**

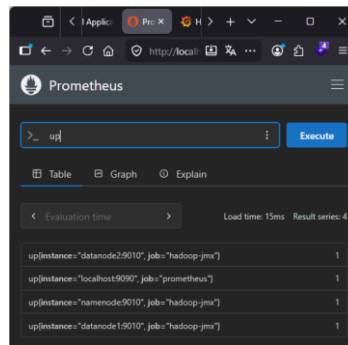


Figura 3.5

### 1.3.3.1. Resultado final (Consultas)

- JMX Exporter* está funcionando correctamente en el *clúster* y *Prometheus* se está comunicando con los nodos con éxito. Pero como la versión del agente que hemos utilizado (*jmx\_prometheus\_javaagent 0.20.0*) no trae la métrica **jmx\_exporter\_up**, en su lugar, vamos a auditar la salud del *JMX Exporter* mediante la métrica estándar de Prometheus filtrando por el job JMX de Hadoop: **up{job="hadoop-jmx"}**

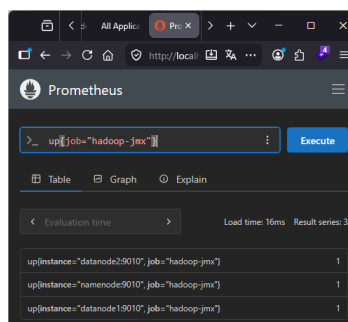


Figura 3.7

Una búsqueda en Google (responde Gemini, evidentemente): “La métrica *jmx\_exporter\_up* dejó de incluirse de forma nativa en las versiones modernas debido a una reestructuración del código del proyecto (donde se estandarizó el uso de *up* y *jmx\_exporter\_build\_info* para evitar redundancias).

*Si quieres forzar la aparición exacta de esa métrica en Prometheus, la versión que debes utilizar es la 0.16.1 (o cualquier versión de la rama 0.15.x hacia atrás). En esas ediciones antiguas, el agente registraba por defecto ese indicador de salud interno."*

2. Tiempo de Prometheus en leer los datos de Hadoop: **jmx\_scrape\_duration\_seconds**

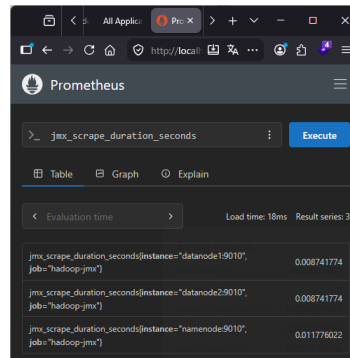


Figura 3.6

3. Frecuencia de ejecuciones del Garbage Collector durante el último minuto:  
**rate(jvm\_gc\_collection\_seconds\_count[1m])**

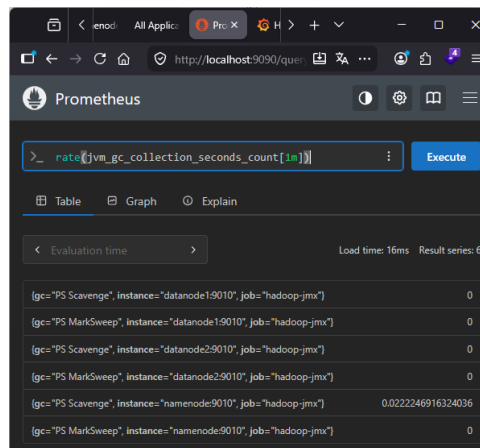


Figura 3.8

### 1.3.4 Crea una visualización [-Ir al índice-](#)

Para esta parte del ejercicio, nos hemos descargado un Dashboard (ver [Fig. 3.9](#)) en concreto para el JMX Prometheus Java Agent. Lo hemos configurado para que se alimente del origen de datos que hemos creado enlazado a Prometheus como se puede ver en la [Fig 3.10](#) y [Fig. 3.11](#).

# Big Data Aplicado

## Máster FP IA & Big Data, Linkiafp.es

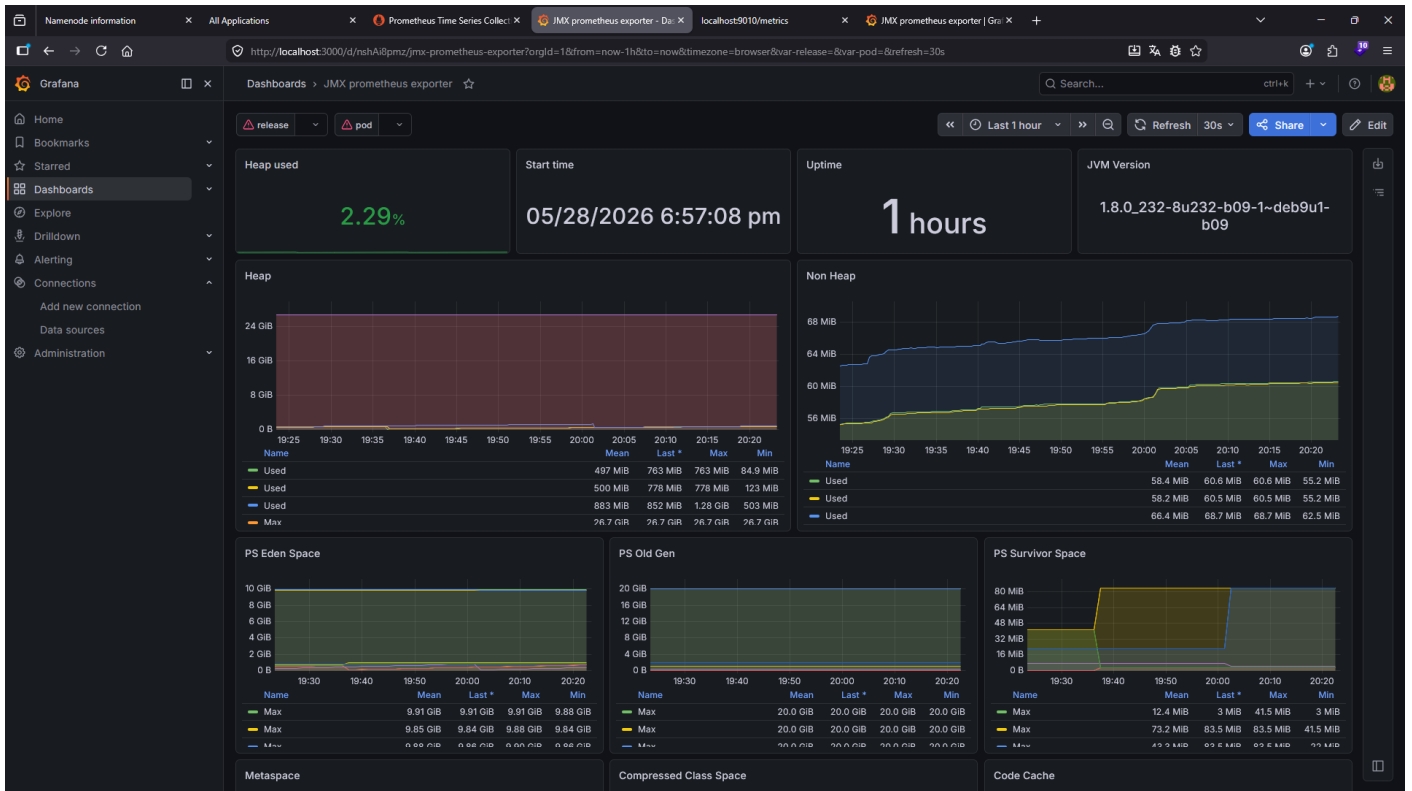


Figura 3.9

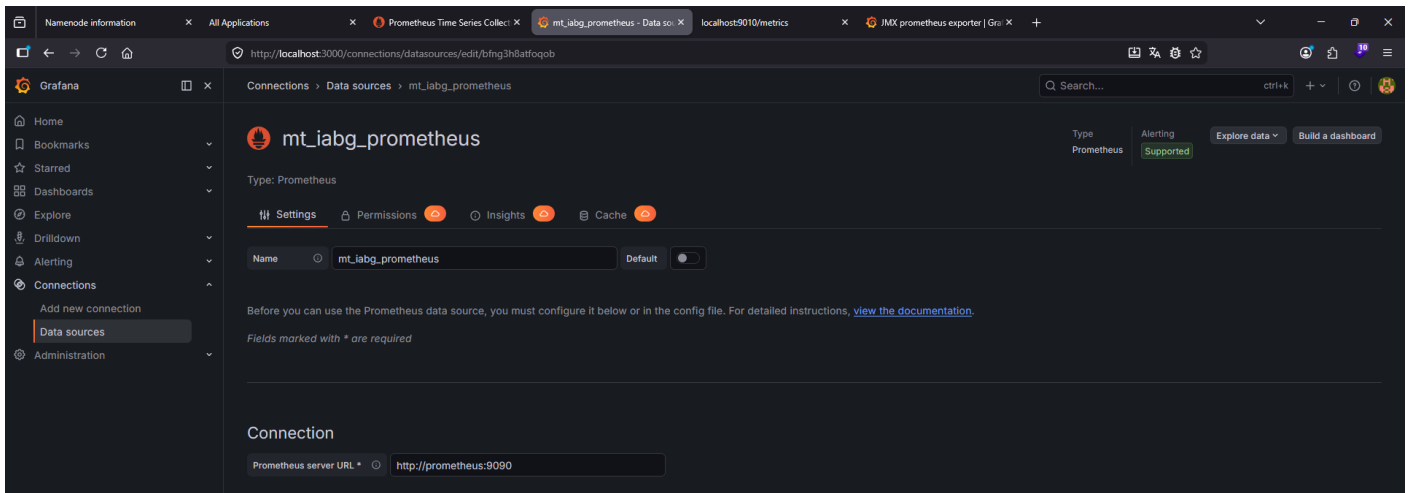


Figura 3.10

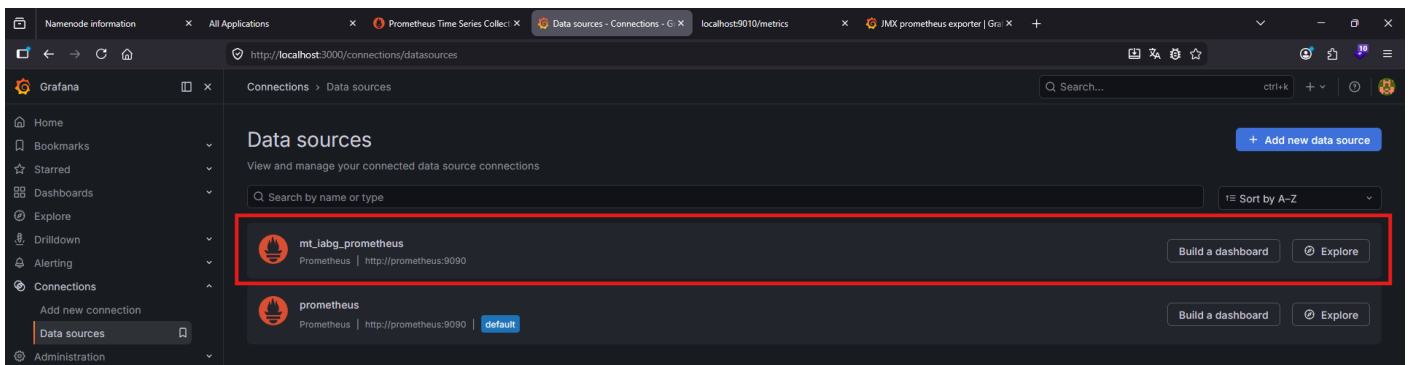


Figura 3.11



A continuación vamos a crear una visualización personalizada, ver [Fig. 3.12](#):

1. Desde el listado de la sección Dashboards, hemos creado un panel dentro de un nuevo cuadro de mandos (Dashboard)
2. Hemos seleccionado el origen de datos (*Data source*) que hemos creado anteriormente con la conexión correcta a Prometheus, en este caso **mt\_iabg\_prometheus** ([Fig. 3.11](#)).
3. La consulta o métrica que hemos seleccionado para el Panel ha sido **jmx\_scrape\_duration\_seconds**, que nos ofrece la telemetría del JMX Exporter.
4. Se ha configurado los metadatos para establecer los textos identificativos para la gráfica
  - **Title:** Tiempo de respuesta
  - **Description:** Tiempo de respuesta del JMX Exporter - Hadoop

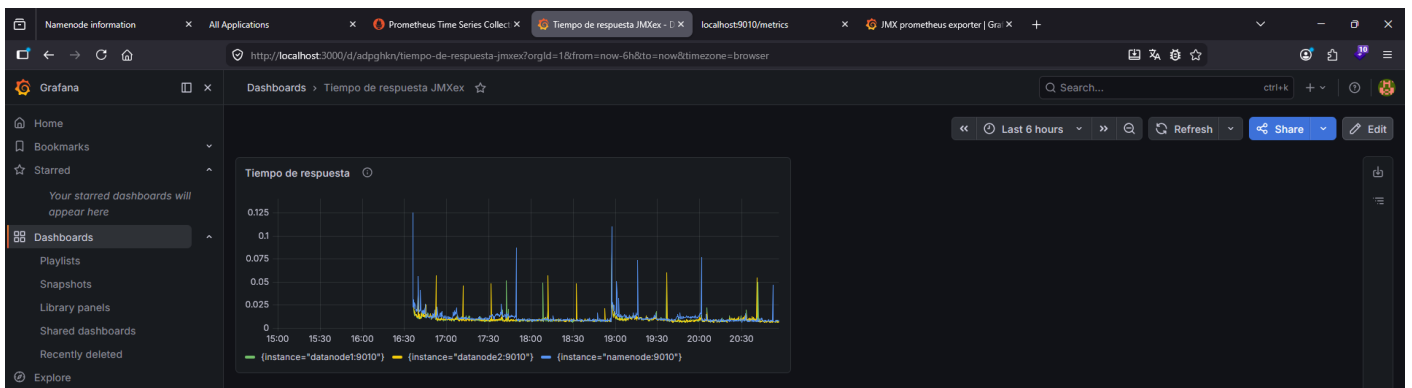


Figura 3.12

### 1.3.5 Configura una alerta [-Ir al índice-](#)

Hemos creado una alerta (ver [Fig. 3.13](#)) que vigile que el JMX Exporter responda de manera rápida y fluida para garantizar que la monitorización del clúster de Hadoop cumpla con esas dos características.

Lo principal que podemos observar en esta alerta es:

- **Qué es lo que mide**, es decir, el tiempo que tarda en recoger la telemetría de los nodos del clúster (NameNode y los dos DataNodes).
- **Qué dispara la alerta**, y para esto se ha configurado que si la consulta supera **1 segundo** durante **1 minuto** continuo, la alerta se activa.
- **Qué impacto tiene** si falla, es decir, que si la JVM está saturada o hay problemas de red en Docker.

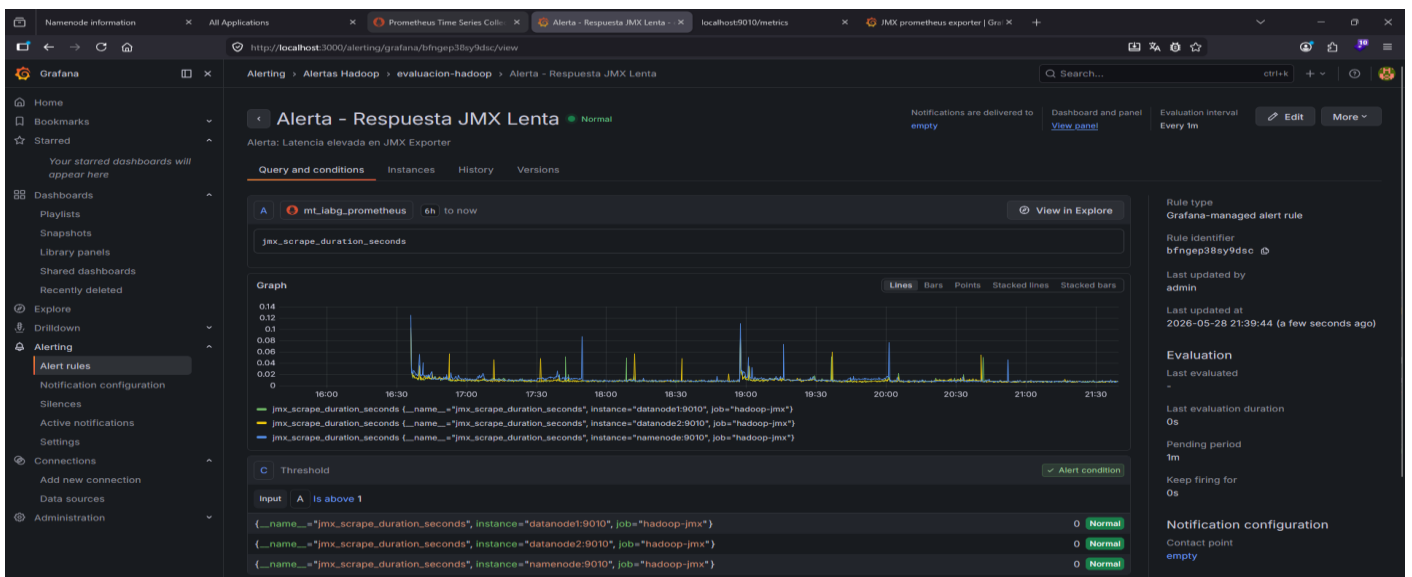


Figura 3.13

### 1.2.3. Anexo Conclusión técnica

Creo que lo más valioso de esta vista es la sección inferior del *Umbral ya que* muestra cómo Grafana está auditando de manera independiente los tres contenedores de Hadoop (el *NameNode* y los dos *DataNodes*). Todas están en estado saludable (Normal) porque sus tiempos de respuesta actuales oscilan en la gráfica en torno a los 0.02 segundos, manteniéndose muy por debajo del límite crítico de 1 segundo que le hemos marcado.

## Parte 4 — Terminal [-Ir al índice-](#)

Como toda la infraestructura es local y se está ejecutando sobre contenedores de Docker Desktop y con integración en WSL, primero debemos "meternos" dentro del contenedor correspondiente para poder lanzar las herramientas de Hadoop, accediendo primeramente al terminal:

```
> docker exec -it namenode bash
root@b58672d7eec8:/# _
```

### 1.4.1. hdfs dfsadmin -report [-Ir al índice-](#)

Una vez dentro, solicitamos al NameNode un informe al sistema de archivos distribuido (HDFS)

```
root@b58672d7eec8:/# hdfs dfsadmin -report
```

Pero la ejecución del comando ha roto:

```
root@b58672d7eec8:/# hdfs dfsadmin -report
Exception in thread "main" java.lang.reflect.InvocationTargetException
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
    at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)
    at java.lang.reflect.Method.invoke(Method.java:498)
    at sun.instrument.InstrumentationImpl.loadClassAndStartAgent(InstrumentationImpl.java:386)
    at sun.instrument.InstrumentationImpl.loadClassAndCallPremain(InstrumentationImpl.java:401)
Caused by: java.net.BindException: Address already in use
    at sun.nio.ch.Net.bind0(Native Method)
    at sun.nio.ch.Net.bind(Net.java:433)
    at sun.nio.ch.Net.bind(Net.java:425)
    at sun.nio.ch.ServerSocketChannelImpl.bind(ServerSocketChannelImpl.java:223)
    at sun.nio.ch.ServerSocketAdaptor.bind(ServerSocketAdaptor.java:74)
    at sun.net.httpserver.ServerImpl.<init>(ServerImpl.java:100)
    at sun.net.httpserver.HttpServerImpl.<init>(HttpServerImpl.java:50)
    at sun.net.httpserver.DefaultHttpServerProvider.createHttpServer(DefaultHttpServerProvider.java:35)
    at com.sun.net.httpserver.HttpServer.create(HttpServer.java:130)
    at io.prometheus.jmx.shaded.io.prometheus.client.exporter.HTTPServer$Builder.build(HTTPServer.java:365)
    at io.prometheus.jmx.common.http.HTTPServerFactory.createHTTPServer(HTTPServerFactory.java:123)
    at io.prometheus.jmx.JavaAgent.premain(JavaAgent.java:60)
    ... 6 more
FATAL ERROR in native method: processing of -javaagent failed
Aborted
root@b58672d7eec8:/# exit
```

Después de investigar (búsqueda en Google y respuesta de Gemini) y aplicar la solución que más adelante detallo, el motivo es que "Cuando ejecutas **hdfs dfsadmin -report**, estás lanzando un *segundo* proceso de Java independiente dentro de ese mismo contenedor para consultar el estado. Como tienes un archivo de configuración .ENV donde defines variables de entorno, en tu archivo **hadoop.env** tienes especificada la variable global de Java de forma genérica **HADOOP\_OPTS**. Al ejecutar este nuevo comando de consola, está intentando cargar también el agente de Prometheus y abrir el mismo puerto. Al ver que el *NameNode* ya lo está usando, salta el **BindException: Address already in use** y aborta."

La solución fue descartar la variable global **HADOOP\_OPTS** usada por todos los contenedores del Clúster de Hadoop y definir, en su lugar, el agente de Java **-javaagent** solo a los servicios específicos que necesitamos monitorizar:

```
# JMX Exporter GLOBAL
# HADOOP_OPTS="-javaagent:/opt/jmx_exporter/jmx_prometheus_javaagent.jar=9010:/opt/jmx_exporter/jmx_config.yml"
#
# JMX Exporter sólo para NameNode
HDFS_NAMENODE_OPTS="-javaagent:/opt/jmx_exporter/jmx_prometheus_javaagent.jar=9010:/opt/jmx_exporter/jmx_config.yml"
# JMX Exporter sólo para DataNodes
HDFS_DATANODE_OPTS="-javaagent:/opt/jmx_exporter/jmx_prometheus_javaagent.jar=9010:/opt/jmx_exporter/jmx_config.yml"
# JMX Exporter para la gestión de recursos (YARN)
YARN_RESOURCEMANAGER_OPTS="-
javaagent:/opt/jmx_exporter/jmx_prometheus_javaagent.jar=9010:/opt/jmx_exporter/jmx_config.yml"
YARN_NODEMANAGER_OPTS="-
javaagent:/opt/jmx_exporter/jmx_prometheus_javaagent.jar=9010:/opt/jmx_exporter/jmx_config.yml"
```

Una vez levantado de nuevo los servicios y haber entrado en la consola del contenedor, ejecutamos de nuevo el comando y obtenemos:

```
root@1947a7c1ce90:/# hdfs dfsadmin -report
Configured Capacity: 2162202353664 (1.97 TB)
Present Capacity: 2020157358080 (1.84 TB)
DFS Remaining: 2020157300736 (1.84 TB)
DFS Used: 57344 (56 KB)
DFS Used%: 0.00%
Replicated Blocks:
  Under replicated blocks: 0
  Blocks with corrupt replicas: 0
  Missing blocks: 0
  Missing blocks (with replication factor 1): 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
Erasure Coded Block Groups:
  Low redundancy block groups: 0
  Block groups with corrupt internal blocks: 0
  Missing block groups: 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0

-----
Live datanodes (2):

Name: 172.19.0.10:9866 (datanode1.mt_iabg_hadoop_cluster_default)
Hostname: datanode1
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 28672 (28 KB)
Non DFS Used: 16030142464 (14.93 GB)
DFS Remaining: 1010078650368 (940.71 GB)
DFS Used%: 0.00%
DFS Remaining%: 93.43%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Fri May 29 13:17:00 UTC 2026
Last Block Report: Fri May 29 13:15:39 UTC 2026
Num of Blocks: 0

Name: 172.19.0.9:9866 (datanode2.mt_iabg_hadoop_cluster_default)
Hostname: datanode2
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
DFS Used: 28672 (28 KB)
Non DFS Used: 16030142464 (14.93 GB)
DFS Remaining: 1010078650368 (940.71 GB)
DFS Used%: 0.00%
DFS Remaining%: 93.43%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 1
Last contact: Fri May 29 13:17:00 UTC 2026
Last Block Report: Fri May 29 13:15:39 UTC 2026
Num of Blocks: 0

root@1947a7c1ce90:/#
```

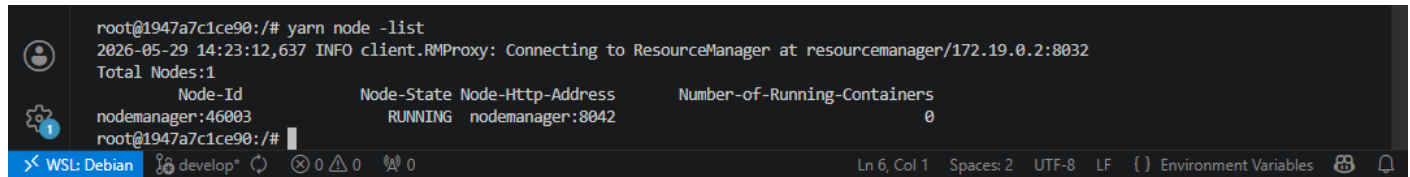
Figura 3.14

Analizando la salida del informe se puede observar que confirma el mismo análisis que hicimos en la [Parte 2, Monitorización Web](#), donde resumiendo se observa que:

- el almacenamiento del Clúster está perfectamente operativo y sin incidencias,
- se detectan los dos DataNodes (datanode1 y datanode2) comunicándose sin problemas,
- ambos nodos suman una capacidad de almacenamiento utilizable real en tu disco de **1.84 TB**,
- que el sistema está completamente limpio, usando sólo **56 KB (0.00%)** y
- que no hay ningún bloque de datos corrupto o perdido.

### 1.4.2. yarn node -list -Ir al índice-

Ejecutando el siguiente comando: **yarn node -list**; en la consola del contenedor **Nodemanager**, obtenemos el siguiente resultado:



```
root@1947a7c1ce90:/# yarn node -list
2026-05-29 14:23:12,637 INFO client.RMProxy: Connecting to ResourceManager at resourcemanager/172.19.0.2:8032
Total Nodes:1
  Node-Id      Node-State  Node-Http-Address  Number-of-Running-Containers
nodemanager:46003  RUNNING    nodemanager:8042      0
root@1947a7c1ce90:/#
```

Figura 3.15

De lo que podemos deducir que:

- la consola conecta con éxito al *ResourceManager* en su IP interna 172.19.0.2.
- se detecta **1 nodo activo** en el clúster asignado al procesamiento (nodemanager:46003).
- el **gestor de recursos (YARN)** está plenamente operativo y sin incidencia aparente.
- se registra **0 contenedores en ejecución**, es decir, el sistema está en reposo ya que no tiene ninguna tarea en ejecución.

### 1.4.3. Configura una alerta -Ir al índice-

Vamos a definir una regla en *Prometheus* para que avise si algún contenedor del clúster se cae de forma inesperada (creo que es lo más obvio en un entorno real, *producción* o *preproducción*)

1. Creamos un archivo **alert.down\_container.yml** en la raíz del proyecto con la siguiente estructura

```
groups:
- name: alertas_hadoop
  rules:
  - alert: ContenedorHadoopCaído
    expr: up == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      summary: "El contenedor {{ $labels.instance }} se ha caído"
      description: "Un servicio del clúster lleva abajo más de 1 minuto."
```

2. Para que Prometheus lo pueda leer, sólo tenemos que añadir la ruta en el archivo de configuración **prometheus.yml** debajo de la sección global, tal y como sigue:

```
global:
  [...]

rule_files:
- "[pointed_path_on_volume_to_rule_file]/alert.down_container"
[...]
```

Una vista rápida a la regla que hemos creado te dice que *se activa sólo cuando la métrica up es 0*, confirmando que un contenedor se ha caído. *Para evitar falsos positivos, el parámetro for: 1m obliga a la espera de un minuto antes de lanzar la alarma.*

Podemos ver en la siguiente captura de pantalla (Fig. 3.16) que la alerta esta creada e inactiva(latente a la espera) a través de la UI de Prometheus en <http://prometheus:9090>

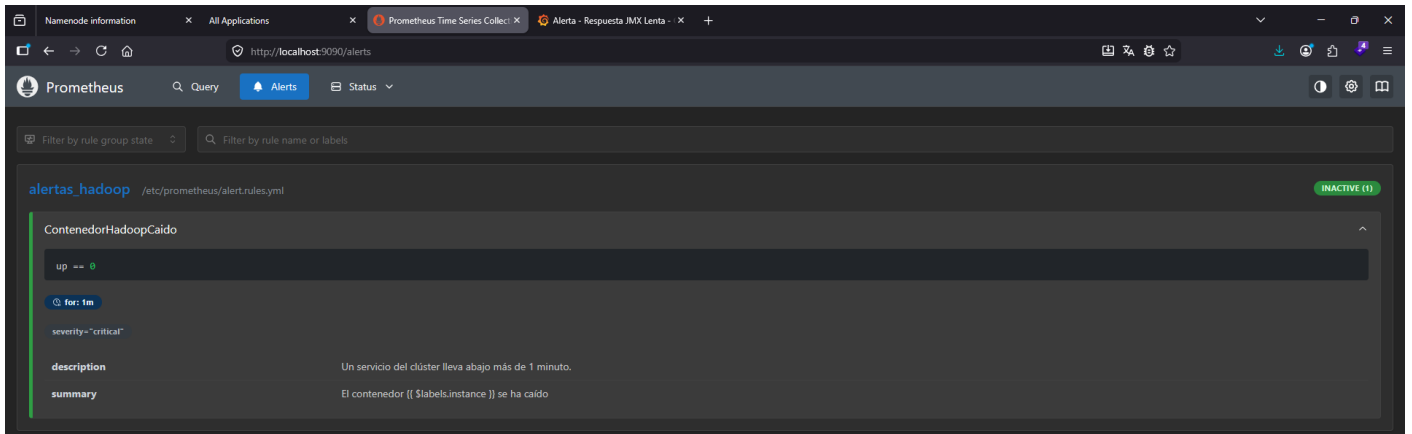


Figura 3.16

Ahora vamos a provocar la caída por ejemplo, del contenedor **datanode2** entrando el comando en la consola:

```
> docker stop datanode2
```

Si a continuación introducimos **docker ps** o, más fácilmente, vemos la ventana de Docker Desktop, el servicio del contenedor aparecerá detenido como se muestra en la imagen siguiente (Fig. 3.17)

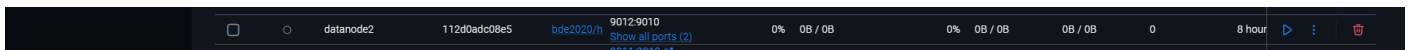


Figura 3.17

Durante los primeros 59 seg. veremos que el panel de Alertas en la UI de Prometheus cambia a **PENDING** (Fig. 3.18)

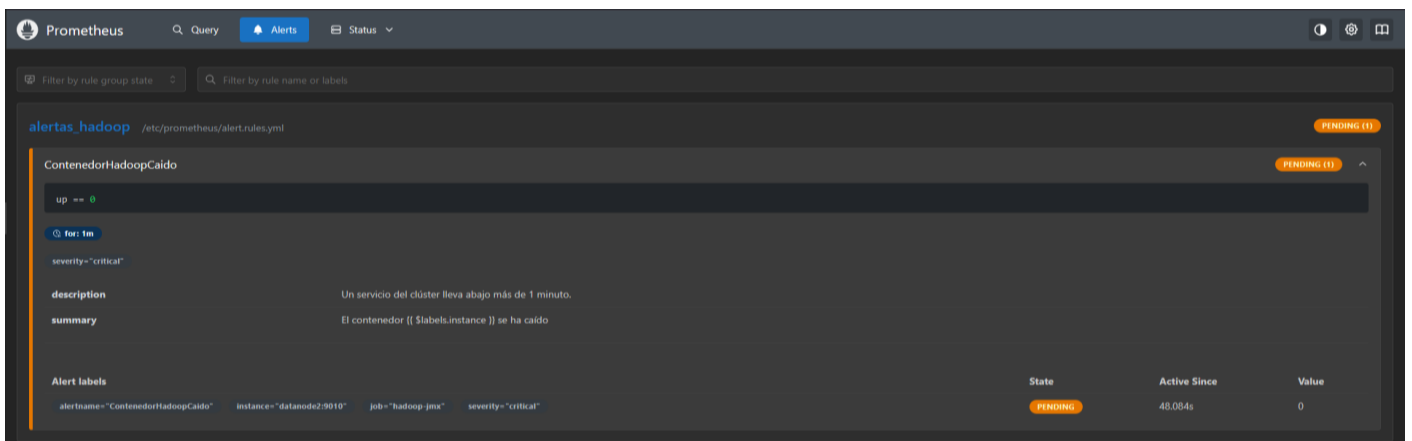


Figura 3.18

Cumplidos 1 minuto debemos ver **FIRING** (Fig. 3.19):

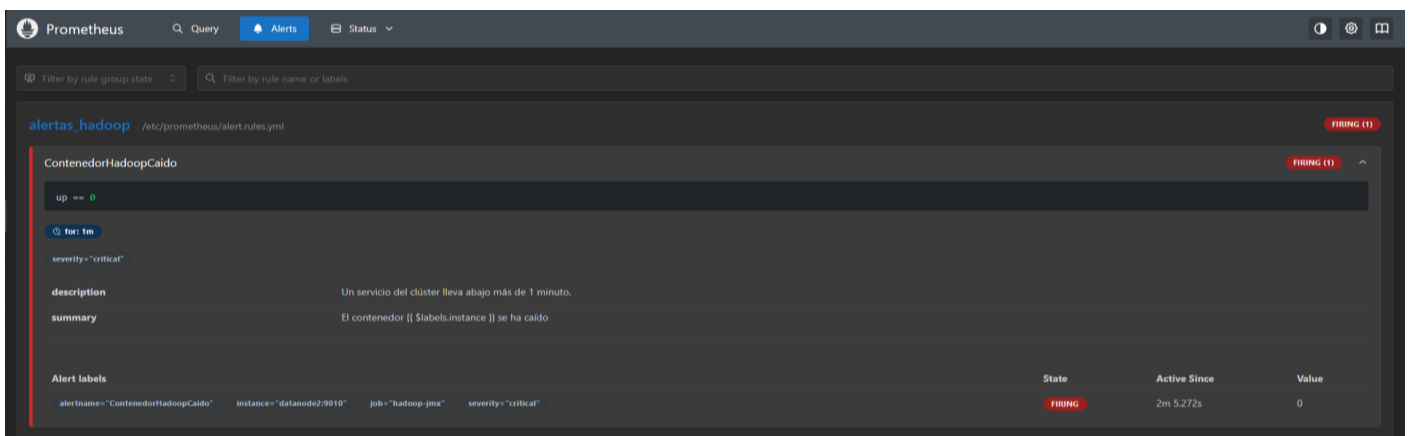


Figura 3.19

## Entrega

- **Capturas de interfaces**

Por cada sección, apartado y/o parte del ejercicio se han añadido todas las imágenes necesarias con la nota a su pie Figura N.x.

- **Explicaciones**

Se han ido explicando las guías paso a paso junto con sus imágenes.

- **Configuración de alerta**

Se han añadido las capturas de pantalla (Grafana) de las alertas y el código de las mismas (Prometheus), y el por qué y para qué de cada paso llevado a cabo.

- **Salidas de comandos**

Se han añadido las capturas de pantalla correspondientes y las explicaciones de cada uno de ellos ejecutado en la consola.

## Ejercicio 2: Análisis y optimización de rendimiento en Hadoop [-Ir al índice-](#)

### Contexto

*Eres el ingeniero de datos de una empresa de comercio electrónico que utiliza Hadoop. Se ha detectado:*

- **Aumento del tiempo de procesamiento de consultas**
- **Problemas en horas pico**
- **Impacto en la toma de decisiones**

### Nota sobre el contexto

Para este caso de estudio que se plantea, existen varios puntos a tener en cuenta. Aunque las preguntas estén orientadas al aspecto técnico, a mi modo de ver, el primero es el factor humano, que, aunque nunca guste abordarlo, a lo largo de toda mi experiencia, creo que es lo primero que hay que hacer.

Lo primero que hay que dejar bastante claro no es sólo que el clúster vaya lento, sino que **las expectativas de negocio están desalineadas con la tecnología elegida**. Si se querían tomar decisiones instantáneas en horas punta basándose en un clúster de Hadoop (diseñado para procesamiento en lote y no para el tiempo real), la arquitectura de base ya nació coja.

Desde el punto de vista de las grandes empresas —y cuando digo grandes, me refiero a grandes y muy, muy grandes—, las inversiones en infraestructura Big Data se hacen a menudo a ciegas, simplemente por moda, novelería o por gestión cosmética. El dinero que se quema del presupuesto asignado es “calderilla de bolsillo” en su escala y es proporcional al error cometido, pero esta pérdida se diluye entre comités y para una PYME o un autónomo, no, ya que esa “calderilla” se traduce en céntimos perdidos que les desmontan el mes.

La conclusión es que *ningún directivo predice el futuro en tiempo real con un panel de control si la infraestructura analítica está en coma; ellos operan basándose en la intuición o en aproximaciones estadísticas y, a menudo, justificando decisiones a toro pasado.*

Todo radica en malas decisiones de diseño que han provocado esta bola de nieve en el rendimiento del clúster.

## 1. Identificación de problemas [-Ir al índice-](#)

Responde:

- **¿Qué problemas de rendimiento puede tener el clúster?**
- **¿Qué factores pueden estar causando estos problemas?**

Analizando el escenario planteado sobre la caída del rendimiento en el comercio electrónico, se me ocurren varios posibles problemas que acarrearán sobre el clúster relacionados con los factores que los provocan, por lo que creo que con el siguiente planteamiento puedo responder a ambas preguntas

### 1.1. Caída por saturación del NameNode.

Resumiendo que NameNode es el centro de toda logística, cuando lleguen las horas punta del comercio, el sistema se saturará y caerá al superar el almacenamiento de, no la información de clientes ni productos, sino de los metadatos de las rutas que tiene en memoria, habiendo sido **previamente definida por diseño arquitectónico de la JVM** y, esto es importante, **sin posibilidad de aumentar a demanda on-premise**.

Por lo que a la conclusión que se llega, es a un mal diseño arquitectónico, que incluso pudiendo haber cumplido y llevado a cabo las reglas y buenas prácticas, la previsión no ha sido correcta, muy seguramente por decisiones basadas en recursos económicos.

Resumiendo en una única línea, el orquestador (NameNode) definido es pequeño.

### 1.2. Cuellos de botella en la E/S en los DataNodes.

El aumento del tiempo de procesamiento puede ser provocado por momentos puntuales, como horas punta y por un aumento orgánico (el negocio tiene un auge estable).

Podemos traducir que los DataNodes son donde físicamente se guardan los bloques de datos, por así decirlo (pero no lo son), los discos duros, así lo entenderá hasta, como yo los llamo un paleta u orangután digital.

Aquí es donde topamos con **la realidad del hardware** y es que en muchos clústeres de Hadoop, siempre por una cuestión de costes, se recortan para utilizar soluciones más baratas y cuando entran miles de registros a escribir, el hardware contratado no puede atender para atender la velocidad de la demanda y llega un momento en el que dice "no puedo leer ni escribir más rápido".

Cuando esto pasa, el servidor se queda literalmente congelado, esperando a que se termine de procesar los datos, pudiendo observar esto mediante gráficas de monitorización, viendo un pico de espera en la E/S.

Resumiendo de nuevo en una única línea, el procesador del servidor está libre, pero el sistema no avanza porque, muy probablemente, el disco es un embudo.

### 1.3. Saturación del ancho de banda de la red.

En el mundo real, en sistemas propios, hablando de grandes empresas claro, la infraestructura construida pasa por tirar cable de fibra y poner tarjetas de red de alta velocidad en servidores, pero esto es costoso y sólo estas empresas pueden permitírselo. Por el contrario, tenemos PYMEs y profesionales autónomos que contratan tecnologías cloud para que no sea tan caro, pero muchos escatiman aquí y contratan planes estándar que se saturan fácilmente.

**¿Qué ocurre en las horas punta?** Cuando se procesan las conversiones, o ventas para que se entienda, llega la fase crítica en la que los nodos necesitan enviarse trozos de datos los unos a los otros para agruparlos, por ejemplo, las ventas de una misma categoría de producto y que se localizan dispersas por todo el clúster).

Durante un pico de tráfico, la infraestructura contratada solo admite hasta cierta cantidad de ancho de banda o, en una gran empresa, esa infraestructura propia, solo puede gestionar el máximo de velocidad a la que los componentes han sido fabricado. Por lo que, igualmente autónomo, pequeña o gran empresa, la red se colapsa. Las conexiones arrojan errores de tiempo de espera y los nodos esclavos, es decir, los DataNodes, se quedan esperando a que les lleguen los datos para poder seguir trabajando.

El clúster se arrastra por saturación de gestión de la información a través de la red alámbrica, inalámbrica, propia o de terceros contratada.

### 1.4. YARN sin recursos.

Si el NameNode es el orquestador y los DataNodes son los discos, YARN, muy posiblemente, es el encargado de la gestión, es decir gestiona los recursos de los que dispone y los distribuye por las tareas que hay abiertas.

En un ejemplo real, en horas punta o subida generalizadas de ventas en el tiempo, todas las tareas, solicitudes, ejecuciones, etc; coinciden a la vez, es decir, todos los procesos automáticos le piden a YARN un trozo de recurso, véase RAM o CPU para poder correr su tarea, pero éste no dispone de más. Las nuevas consultas se quedan congeladas en estado aceptadas (accepted) durante minutos u horas, como si de una cola para entrar en un restaurante se tratase, pero el maître no responde a la demanda.

En horas punta, YARN sufre el colapso del hardware limitado.

### 1.5. Pausas por Recolección de Basura

La máquina virtual de Java (JVM), sobre la que corren Hadoop y Spark, pausa los procesos para limpiar la memoria RAM residual, el recolector de basura (Garbage Collector, GC). Aunque menos probable, pero sí posible, ya que



estamos ante un mal diseño, las pausas congelan temporalmente los nodos durante lo que tarda la limpieza, así que Prometheus puede detectarlos como caídos, aunque erróneamente, y el rendimiento cae en picado.

## 2. Herramientas de monitorización [-Ir al índice-](#)

Responde:

- **¿Qué herramientas usarías? (Hadoop UI, Prometheus, Grafana, logs, etc.)**
- **¿Qué información aporta cada una?**

Para controlar y vigilar los problemas que he nombrado anteriormente, como saturaciones, embudos y pausas críticas, es necesario utilizar un conjunto de herramientas complementarias entre sí, ya que ninguna cubre todo el espectro de información por sí sola.

Con el siguiente enfoque, creo que puedo responder a la cuestión sobre qué herramientas podemos utilizar según la información que nos aporta cada una:

### 2.1. La interfaz web de Hadoop

Es la vista integrada en la propia arquitectura y que ya hemos visto y desglosado, en parte, anteriormente. Por sí misma no guarda históricos de rendimiento, pero nos da una captura en tiempo real del estado de los componentes, como los que siguen:

- **NameNode UI:** Resumiendo toda la sección dedicada a esta utilidad, es donde, de un vistazo, podemos controlar la salud del HDFS, ya que muestra el espacio ocupado, los DataNodes que han dejado de responder y si hay bloques de datos corruptos o perdidos.
- **YARN ResourceManager UI:** De igual forma que antes y haciendo un resumen breve, muestra la memoria RAM y las CPUs virtuales totales frente a las usadas, las colas de ejecución y si hay tareas atascadas.

### 2.2. Prometheus (métricas) y Grafana (históricos)

Es el conjunto perfecto, o al menos el más usado en el ámbito laboral, para monitorizar los componentes a lo largo del tiempo, donde:

- **Prometheus** se encarga de recoger las métricas del clúster en un tiempo dado.
- **Grafana** transforma los resultados obtenidos de las consultas de Prometheus a paneles visuales o gráficas.

### 2.3. Logs

Podríamos definir esto como la “caja negra” real y por excelencia del sistema, y como tal, es el lugar donde rastreamos de forma manual una vez recibido el aviso de que algo ha fallado gracias a las [alertas definidas](#).

Podemos encontrar:

- **Logs de Daemons (hadoop-\*.log):** Donde podemos encontrar errores de conexión entre nodos o denegaciones de permisos en HDFS.
- **Logs de Aplicaciones (yarn-\*.log):** Donde se registra el comportamiento de los procesos de Spark y MapReduce, además de las excepciones de código de los desarrolladores.

## Otras herramientas y utilidades usadas en el ámbito empresarial

Existen otras herramientas, utilidades y aplicaciones muy utilizadas también en entornos corporativos, o que al menos particularmente he usado como son *Nagios* (para monitorización integral y que ofrece también una UI Web y su famoso sistema de alertas), *Zabbix*, *New Relic* y *Cloudera Manager*, entre otras, pero claro, esto es basado en experiencia.

Actualmente, haciendo una búsqueda para ver cuáles son las más populares o, mejor dicho, las más usadas en el entorno laboral de mayor a menor, se encuentran: *Prometheus + Grafana* (esto es evidente), *Datadog*, *Dynatrace*, *New Relic*, *Elastic Stack*, *Splunk*, *Zabbix* y *Nagios*.

## 3. Estrategia de solución [-Ir al índice-](#)

Responde:

- **Pasos para diagnosticar el problema**
- **Cómo optimizar los jobs (MapReduce/Spark)**
- **Medidas para mejorar rendimiento y estabilidad**



Para solucionar de la forma más directa posible, tanto la falta de previsión en el aumento orgánico de las conversiones, como los embudos provocados por posibles picos de horas punta y períodos de ofertas estacionales de referencia, mi propuesta para redactar el informe de diagnóstico sería el siguiente:

### 3.1. Pasos para diagnosticar el problema

1. **Revisar alertas y paneles en Grafana** identificando qué recurso (CPU, RAM, Disco o Red) ha llegado al 100% de su capacidad durante la hora punta o bien desde el período que se ha detectado el deterioro de procesamiento.
2. **Consultar YARN ResourceManager UI** para comprobar si hay acumulación de tareas congeladas en estado y verificar qué procesos están acaparando las colas.
3. **Inspeccionar la NameNode UI:** Verificar el uso de la memoria RAM (Heap) del nodo maestro y revisar si el número total de archivos en el clúster es desproporcionado.
4. **Rastrear los logs**, YARN, Docker o Kubernetes si está contenerizado, del SO, etc; *en busca de excepciones críticas de falta de memoria o hilos bloqueados en los nodos que muestren peores métricas, pero habiendo sido centralizados y analizados por herramientas de observabilidad*, que permitan *buscar patrones de errores mediante filtros temporales* y facilitan la comprensión del problema.

Resumiendo en una única línea, el diagnóstico comienza de lo más general consultando gráficos de Grafana a lo bajo nivel consultando los textos de los logs.

Una vez que tenemos el diagnóstico podemos pasar al Plan de Optimización de Rendimiento:

### 3.2. Cómo optimizar los jobs

- **Particionar los datos** por columna fecha, de esta forma Hadoop crea carpetas físicas en el disco para cada día o mes. Así se **evita que el sistema lea datos innecesarios**, de modo que si estamos en períodos de ofertas estacionales, al buscar por campaña, el clúster va directo a esa carpeta y se salta el resto del año, ahorrando tiempo de disco y red.
- Si el negocio tiene un aumento estable y constante, puede haber tablas que tengan un crecimiento gigantesco y el clúster podría tardar horas al cruzar estas tablas. Con la división de las tablas en un número fijo de ficheros se **mantiene la velocidad de dichos cruces de datos** a largo plazo.
- En horas punta por venta flash **hay que obligar a repartir el trabajo** evitando el desbalanceo de manera que si **añadimos un valor a las claves de los productos**, o incluso un número de lote, se obliga a Hadoop a repartir el procesamiento de forma uniforme entre varios nodos. De esta forma, los datos de los artículos en oferta no irán a parar a la misma máquina del clúster pudiendo llegar a colapsar, mientras que las demás están sin carga.
- **Reducir el peso en la red** mediante un empaquetado de datos más ligero y una agrupación previa en la memoria de cada máquina se elimina el atasco en los cables y se agiliza el procesamiento

### 3.3. Medidas para mejorar rendimiento y estabilidad

- **Programar procesos de unificación** para la ingente cantidad de archivos de logs en bloques más grandes estándar (128 MB o 256 MB), liberando la RAM del NameNode.
- **Segmentar recursos en YARN** configurando el planificador por capacidad asignando una cantidad fija, en porcentaje, para consultas críticas de negocio, aislando el resto para la ingesta de logs.
- **Cambiando el GC** de la JVM de los servidores para usar el recolector de basura que limita el tiempo máximo de parada para evitar congelaciones falsas en el clúster.

## 4. Sistema de alertas [-Ir al índice-](#)

Responde:

- **¿Qué alertas configurarías?**
- **¿Cómo implementarlas? (Prometheus, Grafana, etc.)**

## 4.1 ¿Qué alertas configurarías? [-Ir al índice-](#)

- **Alerta de saturación de NameNode (RAM):** Disparar una alerta si el uso de la memoria supera el 90% durante más de 5 minutos, anticipándonos a un colapso por el problema de la gigantesca cantidad de archivos pequeños de logs.
- **Alerta de embudo en DataNodes:** Lanzar un aviso si el tiempo de espera de disco en cualquier DataNode pasa del 60%, mostrando así el cuello de botella.
- **Alerta de red colapsada:** Configurar una alerta si el tráfico de red de los nodos supera el 85% de su capacidad máxima durante la fase de *Shuffle*.
- **Alerta de atasco en YARN:** Disparar una notificación si el número de aplicaciones en estado congelado (ACCEPTED durante más de un tiempo en minutos) es superior a 5, lo que nos puede delatar que la falta de contenedores libres.
- **Alerta de congelación por JVM (GC Pauses):** Avisar si las pausas por recolección de basura superan los 5 segundos consecutivos, evitando falsos positivos de nodos caídos.

## 4.2. ¿Cómo implementarlas? (Prometheus, Grafana, etc.) [-Ir al índice-](#)

Vamos a implementar la **Alerta de atasco en YARN**, aquí el proceso técnico es bastante directo y se realiza en un par de pasos integrando Prometheus y Grafana de forma nativa.

Como hemos hecho en un [apartado anterior](#), el primer paso consiste en crear un archivo de configuración de reglas dentro del propio servidor de Prometheus (**alert.yarn\_bottleneck.yml**) con las condiciones matemáticas exactas de los dos escenarios críticos que queremos vigilar: *la saturación de red en la fase de Shuffle*.

### 4.2.1. La alerta en Prometheus

```
groups:
- name: yarn_bottleneck
  rules:
  - alert: YARNBottleneck
    expr: sum(yarn_cluster_apps_state{state="ACCEPTED"}) > 5
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "Atasco de aplicaciones en YARN"
      description: "Hay más de 5 aplicaciones retenidas en estado ACCEPTED durante más de 5 minutos por falta de contenedores libres."
```

#### ¿Qué hace?

Vigilamos el indicador de aplicaciones aceptadas. Si el contador supera las 5 aplicaciones de forma continuada durante 5 minutos (el modificador **for**), el sistema entiende que el clúster se ha quedado sin contenedores libres y que los trabajos se están acumulando en cola.

El siguiente paso es enlazarlo dentro del fichero principal de Prometheus, el **prometheus.yml**. Solo hay que abrirlo y añadir la ruta en la sección de reglas:

```
global:
  [...]

rule_files:
- "[pointed_path_on_volume_to_rule_file]/alert.down_container"
- "[pointed_path_on_volume_to_rule_file]/alert.yarn_bottleneck.yml"
[...]
```

Tras reiniciar Prometheus, el motor ya estará evaluando estas consultas en segundo plano de forma constante.

El último paso ocurre ya dentro de Grafana. Al tener Prometheus conectado como origen de datos, diseñamos los paneles visuales para pintar los gráficos de rendimiento del clúster. Para gestionar los avisos sin meter más herramientas en la infraestructura, aprovechamos el motor de alertas nativo de Grafana (Grafana Alerts). Configuramos una política de alertas vinculada a estas mismas métricas y definimos los canales. Si Prometheus detecta que se cumplen los minutos de peligro, la Grafana se encarga de cambiar el color del panel a un rojo de

advertencia y, simultáneamente, dispara una notificación automatizada al instante con el mensaje de error hacia los canales configurados.

## Entrega

Documento en PDF que incluya:

- [Respuestas justificadas](#)

Se han argumentado y razonado las respuestas para cada una de las problemáticas de rendimiento detectadas en el comercio electrónico.

- [Propuesta técnica clara](#)

Se ha diseñado un informe de diagnóstico y plan de acción directo que detalla la arquitectura de soluciones de datos aplicadas como Salting, Partitioning o Bucketing (Cubelado 🗑️).

- [Uso de conceptos vistos en clase](#)

Todo el desarrollo práctico lo fundamento de la forma más ajustada posible y con información encontrada y contrastada en Internet en más de un portal e IA (las búsquedas en Google, las responde Gemini).

## Conclusión

Estos ejercicios permiten trabajar tanto la monitorización práctica como el análisis real de problemas en sistemas Big Data, desarrollando competencias clave en entornos profesionales.